



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Fundamentos Matemáticos para Programación Competitiva
Manual de Apoyo a la Docencia para ICPC

REPORTE DE ACTIVIDAD DOCENTE

QUE PARA OBTENER EL TÍTULO DE

Licenciado en Matemáticas

PRESENTA

Mariana Itzel Melo Tellez

TUTOR

Dr. Manuel Alcántara Juárez



Ciudad Universitaria, Ciudad de México, 2026.

Agradecimientos

Escribe aqui tus agradecimientos.

Índice general

1. Introducción	6
2. Introducción a la Programación Competitiva	7
2.1. ¿Por qué hacer programación competitiva?	9
2.1.1. Beneficios Académicos	9
2.1.2. Oportunidades laborales	10
2.2. Otros concursos y plataformas	10
2.2.1. Ejercicios	10
3. Jueces en Línea	11
3.1. ¿Qué hace un juez en línea?	11
3.1.1. De la propuesta al veredicto	11
3.2. Arquitectura general	12
3.3. ¿Cómo compara las respuestas?	13
3.4. Clarificaciones durante un concurso	13
3.5. Algunos jueces y plataformas en línea	14
3.6. Una observación importante	14
4. Entrada/Salida	15
4.1. Primer contacto con la entrada y la salida	15
4.2. Entrada estándar, salida estándar y funciones dadas	15
4.3. Patrones comunes de entrada	16
4.3.1. Unicaso	16
4.3.2. Multicaso específico	16
4.3.3. Multicaso controlado por valores	17
4.3.4. Multicaso controlado por EOF	17
4.3.5. Multicaso con salida numerada	18
4.3.6. Multicaso con datos en una línea	18

4.3.7. Opción de función	19
4.4. Una observación práctica	19
5. Álgebra	20
5.1. Recurrencias	22
5.1.1. Solucion DP	22
5.1.2. Solución Exponenciación de Matrices	22
5.2. Polinomios e interpolación de Lagrange	23
5.3. Transformada de Fourier rápida (FFT) y NTT	26
5.4. Problemas	29
6. Teoría de números	30
6.1. Bases	30
6.2. Aritmética modular	31
6.3. Números primos y criba de Eratóstenes	32
6.4. Teorema de Bézout	33
6.5. Pequeño teorema de Fermat	33
6.6. Inverso multiplicativo modular	34
6.7. Congruencias lineales y Teorema Chino del Residuo	35
6.8. Funciones multiplicativas	36
6.9. Logaritmo discreto y Baby-step Giant-step	40
6.10. Ejercicios	41
7. Combinatoria y conteo	43
7.1. Bases	43
7.2. Permutaciones	44
7.3. Combinaciones	45
7.4. Principio de Inclusión-Exclusión	46
7.5. Barras y estrellas	48
7.6. Lema de Burnside	48
7.6.1. Órbitas y puntos fijos	49
7.7. Números de Catalan	50
7.8. Números de Stirling de segundo tipo	51
7.9. Números de Bell	51
7.10. Problemas	52

8. Geometría computacional	53
8.1. Bases	54
8.2. Intersecciones	55
8.3. Polígonos	56
8.4. Convex Hull	58
8.5. Algoritmos de barrido	60
8.6. Ejercicios	64
8.7. Problemas	64

Capítulo 1

Introducción

Este manual es una guía de apoyo para fortalecer la preparación matemática en programación competitiva, con enfoque en el Concurso Internacional Universitario de Programación (ICPC). Está dirigido tanto a estudiantes de matemáticas que buscan iniciarse en esta área como a participantes que desean consolidar y ampliar sus herramientas teóricas.

Si bien existen libros de programación competitiva ampliamente reconocidos, gran parte de ellos se concentra en algoritmos, implementación y estructuras de datos. Este manual busca complementar ese panorama con un enfoque explícito en la formación matemática que requiere la competencia y con una propuesta adaptada al contexto académico de la Facultad de Ciencias.

Además de la exposición conceptual, el manual incluye un conjunto de problemas cuidadosamente seleccionados y también problemas de autoría propia, diseñados para reforzar de manera directa los conocimientos presentados en cada tema.

La organización del contenido responde a áreas temáticas, no a niveles de dificultad, con el fin de que cada lector avance de acuerdo con sus intereses y necesidades. En el Capítulo 2 se presenta una introducción general a la competencia ICPC, su dinámica y sus reglas principales; en el Capítulo 3 se abordan los fundamentos de entrada/salida y el uso de jueces en línea; y en el Capítulo 4 se desarrolla el bloque de teoría de números aplicado a resolución de problemas.

Al final del documento se incluyen las editoriales de los problemas presentados. Estas editoriales comienzan con *hints* para orientar el proceso de resolución y no muestran de inmediato la solución completa. Para obtener mejores resultados, se sugiere seguir este orden de trabajo: primero intentar resolver cada problema por cuenta propia, después apoyarse en los *hints* cuando sea necesario y, finalmente, consultar la editorial para comparar enfoques y comprender la solución propuesta.

Capítulo 2

Introducción a la Programación Competitiva

La programación competitiva suele describirse como un deporte mental: exige constancia, disciplina y entrenamiento estructurado. El objetivo no es solamente encontrar una solución correcta, sino hacerlo con precisión y en el menor tiempo posible.

Su filosofía se centra en la resolución eficiente de problemas clásicos de ciencias de la computación. En este contexto, los enunciados están diseñados para ser resolubles con técnicas y algoritmos conocidos; es decir, no se trata de problemas de investigación abierta, sino de retos que evalúan la capacidad de modelar, implementar y optimizar soluciones bajo presión de tiempo. Resolver un problema implica comprender el enunciado, elegir una estrategia algorítmica adecuada y traducirla a código en un lenguaje de programación. El componente competitivo aparece cuando, además de resolver correctamente, se debe hacerlo rápido y con la menor cantidad de intentos fallidos.

En comparación con hackatones u otros concursos de desarrollo, donde se privilegia la construcción de productos completos durante varios días, la ICPC y competencias similares se enfocan en problemas puntuales con criterios estrictos de correctitud, tiempo y memoria. El sitio oficial de la ICPC la describe como una competencia algorítmica universitaria. En la práctica, esto significa que cada solución enviada es evaluada automáticamente por un programa llamado juez en línea que verifica si el programa responde correctamente a todos los casos de prueba y respeta los límites establecidos. Esta dinámica es una diferencia clave frente a concursos con evaluación más abierta y en ocasiones algo subjetiva, como los hackatones.

La ICPC es actualmente la competencia universitaria más representativa en esta disciplina. Se compite en equipos de tres estudiantes de la misma institución y, según la región, el proceso se organiza en varias fases clasificatorias. En el caso de México, desde 2024 se introdujo una estructura de **cuatro etapas**; entre ellas, el Gran Premio de México, compuesto por tres fechas de competencia clasificatorias. En cada fecha se resuelve un conjunto de problemas, usualmente entre 9 y 13, en tiempo limitado y la clasificación considera tanto la cantidad de problemas resueltos como la penalización acumulada por tiempo e intentos incorrectos.

Para hacerlo más claro e ilustrativo, tomemos como ejemplo el problema clásico de Codeforces 231A.

A. Team

time limit per test: 2 seconds

memory limit per test: 256 megabytes

One day three best friends Petya, Vasya and Tonya decided to form a team and take part in programming contests. Participants are usually offered several problems during programming contests. Long before the start the friends decided that they will implement a problem if at least two of them are sure about the solution. Otherwise, the friends won't write the problem's solution. This contest offers n problems to the participants. For each problem we know which friend is sure about the solution. Help the friends find the number of problems for which they will write a solution.

Input

The first input line contains a single integer n ($1 \leq n \leq 1000$), the number of problems in the contest. Then n lines contain three integers each, each integer is either 0 or 1. If the first number equals 1, then Petya is sure about the problem's solution; otherwise he isn't sure. The second number shows Vasya's view, and the third number shows Tonya's view.

Output

Print a single integer: the number of problems the friends will implement on the contest.

Example

Input

```
3
1 1 0
1 1 1
1 0 0
```

Output

```
2
```

Como se observa en este ejemplo, usualmente todo problema de programación competitiva consta de las siguientes secciones:

- **Tiempo límite:** tiempo máximo de ejecución permitido.
- **Memoria límite:** memoria RAM máxima para la solución.
- **Descripción:** contexto y tarea específica a resolver.
- **Input (entrada):** formato exacto de los datos de entrada.
- **Output (salida):** formato exacto de la respuesta esperada.
- **Ejemplos:** casos de prueba para validar la interpretación.

Una pregunta valiosa para quien inicia es: *¿entiendo lo que me pide el problema?*; y después, *ya que entendí el problema, ¿puedo crear un código para resolverlo?*; finalmente, *¿cuánto tiempo me tomaría escribir un código correcto y sin errores?*. Esta autoevaluación ayuda a medir el progreso real en programación competitiva y, sobre todo, a construir confianza con cada intento. Incluso cuando la primera solución no sale a la perfección, cada ajuste mejora la lectura de problemas, la precisión del código y la velocidad de respuesta en concurso. Entre más problemas se resuelven, mayor "condición" se desarrolla para identificar patrones, tipos de enunciados, técnicas y trucos frecuentes; en consecuencia, cada vez resulta más natural acercarse a una solución correcta desde el primer intento.

La idea central para resolver el problema 231A es contar, en cada línea, cuántos valores son iguales a 1; si la suma es mayor o igual a 2, se incrementa el contador de problemas resueltos. Esta tarea permite practicar lectura de entrada, recorridos simples y condiciones lógicas. El código puede implementarse de forma directa:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     int implementados = 0;
8
9     for (int i = 0; i < n; i++) {
10         int a, b, c;
11         cin >> a >> b >> c;
12
13         // Si al menos dos están seguros, se implementa el problema.
14         if (a + b + c >= 2) {
15             implementados++;
16         }
17     }
18
19     cout << implementados << "\n";
20 }
```

En este ejercicio también queremos exponer un detalle importante: en la salida se imprime "\n". Aunque parezca menor, el juez en línea suele comparar texto de forma estricta. Si el formato no coincide exactamente con lo esperado (por ejemplo, cuando el resultado debe ir en una línea y no se respeta), es posible recibir un veredicto **Wrong Answer** aun cuando la lógica sea correcta. Iniciar puede ser frustrante, pero después de superar esa curva de aprendizaje, este deporte se vuelve sumamente reconfortante.

2.1. ¿Por qué hacer programación competitiva?

2.1.1. Beneficios Académicos

La programación competitiva aporta beneficios académicos importantes para la formación en ciencias de la computación. En primer lugar, fortalece habilidades esenciales como el pensamiento algorítmico, el razonamiento lógico y la resolución eficiente de problemas. Al enfrentarse a ejercicios desafiantes, el estudiante aprende a descomponer tareas complejas en partes más pequeñas, detectar patrones y diseñar estrategias correctas y eficientes.

Además, funciona como un puente entre la teoría y la práctica. Conceptos vistos en clase –como estructuras de datos, análisis de complejidad, teoría de números o técnicas de optimización– dejan de ser abstractos cuando deben aplicarse para resolver problemas concretos bajo restricciones reales de tiempo y memoria.

Otro beneficio clave es que fomenta el aprendizaje continuo. Cada problema nuevo expone al estudiante a ideas distintas y lo motiva a investigar enfoques alternativos, comparar soluciones y refinar su forma de pensar. Con el tiempo, esto fortalece hábitos académicos valiosos: validar casos límite, justificar la correctitud de una solución y mejorar progresivamente la claridad del código.

2.1.2. Oportunidades laborales

La práctica constante en programación competitiva también tiene impacto directo en el perfil profesional. Muchas empresas tecnológicas evalúan a sus candidatos mediante entrevistas técnicas centradas en resolución algorítmica de problemas; por ello, entrenar este tipo de pensamiento representa una ventaja real en procesos de selección para pasantías y puestos de tiempo completo.

Durante estas evaluaciones no solo importa llegar a la respuesta correcta. También se observa cómo razona la persona candidata, cómo comunica sus ideas, cómo depura errores, cómo justifica la complejidad de su solución y cómo se adapta cuando cambian las condiciones del problema. Todas estas competencias se trabajan de forma natural en competencias de programación.

Finalmente, la experiencia en concursos también puede aportar visibilidad profesional: buenos resultados, constancia y participación activa en comunidades técnicas suelen reflejar iniciativa, disciplina y capacidad de trabajo intelectual bajo presión, cualidades altamente valoradas en la industria.

2.2. Otros concursos y plataformas

Además de la ICPC, existen competencias en las que puede participar cualquier persona, sin necesidad de ser estudiante. Algunos ejemplos son Google Hash Code, Google Kick Start y Mexico ICPC Masters.

También hay plataformas que organizan concursos de práctica de forma regular, como Codeforces y AtCoder. Estos espacios son ideales para entrenar de manera constante, comparar enfoques con otras personas y acostumbrarse a resolver problemas bajo presión de tiempo.

En resumen, existen muchos jueces en línea y miles de problemas disponibles para seguir practicando. Siempre habrá un nuevo reto esperándote.

2.2.1. Ejercicios

- Tabla para Pastel (omegaUp): Concurso FCIencias 2016.
- Watermelon (Codeforces 4A): Determinar si una sandía de peso w puede dividirse en dos partes pares positivas.
- Minimize (Codeforces 2009A): Para cada caso, hallar el mínimo de $(c - a) + (b - c)$ con $a \leq c \leq b$.
- I Wanna Be the Guy (Codeforces 469A): Decidir si dos jugadores, combinando los niveles que cada uno puede pasar, cubren todos los niveles del juego.

Capítulo 3

Jueces en Línea

Los jueces en línea son plataformas que reciben el código fuente de una solución, lo compilan, lo ejecutan con casos de prueba predefinidos y determinan automáticamente si el programa cumple con lo pedido en el problema. En programación competitiva, prácticamente toda la evaluación pasa por este tipo de sistemas.

3.1. ¿Qué hace un juez en línea?

Cuando una persona concursante envía una solución, el juez no revisa si “la idea parece correcta”, sino si el programa produce exactamente el comportamiento esperado. Para ello, utiliza un conjunto de casos de prueba preparados por quienes diseñaron el problema, junto con una salida correcta previamente validada.

Antes de que un problema llegue al concurso, normalmente pasa por una etapa de preparación. En muchas competencias existe una convocatoria o proceso interno para proponer problemas; cada propuesta debe venir acompañada de una solución conocida, correcta y justificable. En otras palabras, no se plantean problemas de investigación abierta ni preguntas cuya solución todavía sea incierta: el comité necesita saber de antemano que el problema se puede resolver y bajo qué complejidad.

Además del enunciado, quienes preparan el problema suelen entregar varios elementos técnicos: una o más soluciones de referencia, un conjunto de archivos de entrada con sus salidas esperadas, límites de tiempo y memoria, y en algunos casos un comparador especial. Estas piezas permiten construir la evaluación automática con suficiente confianza. Si el problema admite varias respuestas correctas, tolerancias numéricas o criterios más sutiles que una comparación literal, entonces el comparador también forma parte esencial del material del problema.

En muchos casos, los archivos de prueba no se escriben uno por uno de manera artesanal, sino que se generan a partir de generadores y después se validan con soluciones de referencia. El objetivo es cubrir instancias pequeñas, casos borde y situaciones diseñadas para detectar errores frecuentes o algoritmos demasiado lentos. Así, el juez no solo revisa si una solución funciona en ejemplos sencillos, sino si se comporta correctamente en escenarios representativos y exigentes.

3.1.1. De la propuesta al veredicto

Una vez preparado el problema, el juez en línea sigue un proceso bien definido para evaluar cada envío. De manera general, recibe el programa de la persona concursante, lo compila si el lenguaje lo requiere,

lo ejecuta con los casos de prueba ocultos y compara la salida producida con la respuesta esperada o con el criterio establecido por el comparador. Si además se respetan las restricciones de tiempo y memoria, la solución se acepta. De forma detallada, este proceso puede descomponerse en varias etapas:

1. El concursante escribe su programa y lo envía a la plataforma.
2. El juez recibe el archivo y verifica que el envío sea válido para el lenguaje seleccionado.
3. Si el lenguaje lo requiere, el sistema compila el programa.
4. Si la compilación tiene éxito, el evaluador ejecuta la solución sobre los casos de prueba ocultos.
5. Para cada caso, el programa recibe una entrada determinada y produce una salida.
6. Esa salida se compara con la respuesta esperada, ya sea de forma estricta o mediante un comparador especial.
7. Al mismo tiempo, el juez controla el tiempo de ejecución, el uso de memoria y, en algunas plataformas, el volumen de salida.
8. Si todos los casos pasan correctamente y se respetan las restricciones, la solución recibe el veredicto **Accepted**.

Como resultado de este proceso, el juez devuelve un veredicto. Los más frecuentes son los siguientes:

- **Accepted (AC):** la solución produjo la respuesta correcta dentro de los límites establecidos.
- **Wrong Answer (WA):** la salida no coincide con la esperada o no satisface el criterio del checker.
- **Compilation Error (CE):** el programa no compiló correctamente.
- **Run Time Error (RTE):** ocurrió un error durante la ejecución, como división entre cero o acceso inválido a memoria.
- **Time Limit Exceeded (TLE):** la solución tardó más de lo permitido.
- **Memory Limit Exceeded (MLE):** la solución usó más memoria de la disponible.
- **Output Limit Exceeded (OLE):** el programa imprimió más salida de la permitida.
- **Presentation Error (PE):** en algunas plataformas, la respuesta es esencialmente correcta pero el formato no coincide; en otras, este caso se reporta simplemente como **WA**.

3.2. Arquitectura general

Aunque la implementación concreta puede variar entre plataformas, un juez en línea suele estar compuesto por varias partes que colaboran entre sí. En una versión un poco más realista, el envío pasa primero por un frontend, luego se registra como una solicitud de evaluación y entra en una cola. A partir de esa cola, uno o varios procesos trabajadores, o *workers*, toman envíos pendientes, los compilan y los ejecutan dentro de un entorno aislado. Después, la salida producida se envía a un comparador, que decide si la respuesta debe aceptarse o no.

Una forma simplificada de visualizarlo es la siguiente:

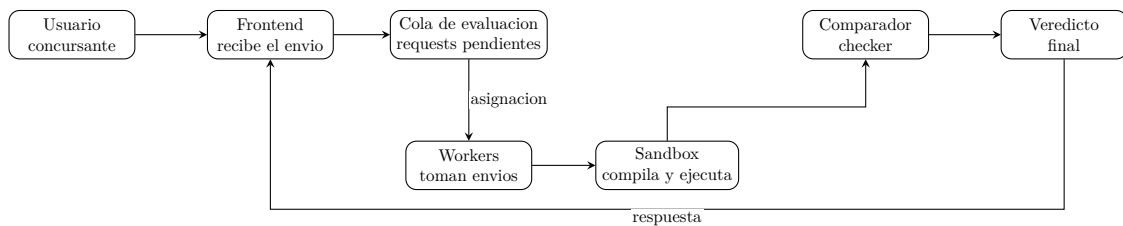


Figura 3.1: Esquema simplificado del flujo de evaluación en un juez en línea.

En este esquema, el **usuario o concursante** prepara y envía su solución; el **frontend** recibe ese envío y valida aspectos básicos antes de registrarlo; la **cola de evaluación** conserva los trabajos pendientes; los **workers** toman envíos de esa cola y coordinan su procesamiento; el **sandbox** compila y ejecuta el programa dentro de un entorno aislado y con recursos limitados; el **comparador o checker** contrasta la salida producida con la salida esperada, y en algunos casos puede ignorar diferencias de mayúsculas, normalizar texto o aplicar tolerancias numéricas; finalmente, el sistema devuelve un **veredicto** al usuario.

3.3. ¿Cómo compara las respuestas?

El punto más delicado del proceso suele ser la comparación entre la salida esperada y la salida producida por el programa. En muchos problemas, esta comparación es estricta: el juez revisa carácter por carácter, por lo que una diferencia mínima en espacios, saltos de línea o formato puede hacer que una respuesta correcta en idea sea rechazada.

Sin embargo, no todos los problemas usan exactamente el mismo criterio. Existen al menos tres escenarios comunes:

- **Comparación estricta:** la salida debe coincidir exactamente con el formato esperado.
- **Comparación con tolerancia:** se permiten pequeñas diferencias, por ejemplo en problemas con números reales donde se acepta cierto error absoluto o relativo.
- **Comparación por equivalencia:** el juez acepta múltiples salidas válidas, por ejemplo cuando los elementos pueden aparecer en cualquier orden o cuando se usa un checker especial que verifica propiedades de la respuesta y no solo texto literal.

Por esta razón, leer con cuidado la sección de salida del problema es tan importante como diseñar el algoritmo. Muchas soluciones fallan no por un error matemático o lógico, sino por no respetar exactamente lo que el problema solicita imprimir.

3.4. Clarificaciones durante un concurso

En competencias oficiales, el juez en línea no trabaja de manera aislada: suele existir un comité de personas organizadoras que supervisa el comportamiento de la plataforma y atiende dudas sobre los enunciados. Si una persona concursante detecta una posible ambigüedad o un error, puede enviar una aclaración para que el comité revise el caso.

Si se confirma un problema en el enunciado, en los datos o en el criterio de evaluación, el comité puede corregirlo y volver a juzgar las soluciones enviadas. En situaciones más delicadas, incluso puede

retirarse un problema del concurso. Esto muestra que, aunque el juez automatiza la evaluación, el diseño y mantenimiento del concurso sigue dependiendo de una supervisión humana cuidadosa.

3.5. Algunos jueces y plataformas en línea

Existen muchas plataformas en las que es posible practicar, competir o simplemente familiarizarse con el estilo de evaluación automática. Cada una tiene su propio enfoque, comunidad y tipo de problemas. A continuación se mencionan algunas de las más conocidas:

- **NetCode:** es una plataforma orientada a la práctica de programación y evaluación automática. Puede ser útil para entrenar con problemas y acostumbrarse al flujo básico de envío y veredicto. Se puede consultar en <https://netcode.dev>.
- **LeetCode:** es una plataforma muy popular para practicar problemas algorítmicos, especialmente entre quienes se preparan para entrevistas técnicas. Cuenta con una modalidad de paga que da acceso, entre otras cosas, a editoriales y materiales adicionales. Su dirección es <https://leetcode.com>.
- **Kattis:** es ampliamente utilizado tanto para práctica individual como en competencias universitarias. Su colección de problemas es extensa y su estilo de enunciados se parece mucho al de varios concursos formales. Puede consultarse en <https://open.kattis.com>.
- **omegaUp:** es una plataforma muy importante en el ámbito hispanohablante y particularmente en México. Permite practicar, organizar concursos y seguir rankings, por lo que ha sido una puerta de entrada para muchas personas que comienzan en programación competitiva. Su sitio es <https://omegaup.com>.
- **Codeforces:** es una de las plataformas más activas de la comunidad internacional de programación competitiva. Ofrece concursos frecuentes, problemas clasificados por dificultad, editoriales y una comunidad muy dinámica. Puede consultarse en <https://codeforces.com>.

Explorar varias plataformas también ayuda a desarrollar flexibilidad, ya que no todas presentan los problemas del mismo modo ni usan exactamente el mismo estilo de juez. Con el tiempo, familiarizarse con estas diferencias hace más natural adaptarse a concursos y entornos diversos.

3.6. Una observación importante

Para quienes comienzan, uno de los mayores puntos de frustración es descubrir que un programa puede fallar aun cuando la idea general sea correcta. Un simple espacio extra, una línea faltante o un formato diferente al solicitado puede ser suficiente para recibir un veredicto negativo.

Con la práctica, esta parte deja de parecer arbitraria y empieza a formar parte natural del proceso de resolución. Aprender a leer con precisión, imprimir exactamente lo pedido y revisar cuidadosamente la salida es también parte del entrenamiento en programación competitiva.

Capítulo 4

Entrada/Salida

4.1. Primer contacto con la entrada y la salida

Lo primero que una persona suele aprender al hacer programación competitiva es manejar correctamente la entrada y la salida de un problema. Recordemos que los jueces en línea ejecutan nuestro programa y le proporcionan texto que corresponde a la entrada; ese texto puede venir en varios formatos, y por ello es importante estar atento a la manera en que se presentan los datos.

La mayoría de los problemas trabajan con **entrada estándar** y **salida estándar**. Esto significa que el programa recibe los datos por medio de un flujo de entrada, por ejemplo `cin` en C++, y escribe su respuesta mediante un flujo de salida, asociado con `cout`. En un concurso no se acostumbra abrir archivos manualmente; el juez se encarga de proporcionar la entrada al programa y de capturar su salida para revisarla.

Este estilo aparece con mucha frecuencia en concursos formales, donde justamente parte del trabajo consiste en leer correctamente la entrada y producir la salida con el formato exacto que pide el enunciado. En ese sentido, no basta con tener una buena idea: también hace falta traducirla a un programa que interprete bien los datos desde el primer momento.

4.2. Entrada estándar, salida estándar y funciones dadas

En muchos concursos y jueces clásicos, leer la entrada es parte explícita del problema. El programa debe reconocer cuántos datos vienen, en qué orden aparecen y cómo termina cada caso. Esto forma parte del entrenamiento, porque obliga a prestar atención no solo al algoritmo sino también al formato.

Sin embargo, existen plataformas en las que este paso se simplifica. En algunos jueces en línea se proporciona una función ya definida y lo único que se espera es completar su implementación. En esos casos, el propio juez se encarga de interpretar la entrada, llamar a la función con los parámetros adecuados y revisar el resultado devuelto. Esto hace que la curva de aprendizaje inicial sea un poco más amable, pues ya no es necesario lidiar desde el principio con toda la parte de lectura y escritura.

Ninguno de estos estilos es “mejor” que el otro: simplemente responden a objetivos distintos. Los concursos formales suelen exigir dominio de entrada y salida completas, mientras que otras plataformas buscan facilitar el enfoque directo sobre la lógica del problema.

4.3. Patrones comunes de entrada

Una de las habilidades más útiles al comenzar es reconocer rápidamente qué tipo de entrada se está usando. A continuación se presentan algunos patrones frecuentes, junto con ejemplos pequeños en C++.

4.3.1. Unicaso

En este patrón, hay un único caso de prueba cada vez que el juez ejecuta el programa. Se leen los datos una sola vez, se procesa la información y se imprime la respuesta.

Por ejemplo, supongamos que la entrada se compone de un número, seguido de un espacio, seguido de otro número y finalmente un salto de línea.

Entrada ejemplo

50 100

Salida ejemplo

150

En C++, la instrucción `cin >> a >> b;` ignora automáticamente los espacios y saltos de línea entre enteros, por lo que no hace falta procesarlos de forma manual.

```
1 int main() {  
2     int a, b;  
3     cin >> a >> b;  
4     cout << a + b << "\n";  
5 }
```

La misma idea puede verse con `scanf`, que también ignora espacios en blanco entre números cuando se usa un formato como `"%d%d"`.

```
1 int main() {  
2     int a, b;  
3     scanf("%d %d", &a, &b);  
4     printf("%d\n", a + b);  
5 }
```

4.3.2. Multicaso específico

Aquí el número de casos de prueba aparece en la primera línea de la entrada. Después de leer ese número, se repite el mismo proceso para cada caso.

Por ejemplo, si se tienen tres casos y en cada uno se pide sumar dos números, la entrada podría ser

Entrada ejemplo

3
1 2
10 20
50 100

Salida ejemplo

3
30
150

En este caso, primero se lee cuántos casos hay y luego se repite el mismo bloque de lectura y escritura ese número de veces.

```
1 int main() {  
2     int t;  
3     cin >> t;  
4  
5     while (t--) {
```



```

6         int a, b;
7         cin >> a >> b;
8         cout << a + b << "\n";
9     }
10 }

```

4.3.3. Multicaso controlado por valores

En algunos problemas no se especifica cuántos casos hay, sino que se da una condición de parada. Un ejemplo clásico es seguir leyendo pares a , b hasta encontrar $a = 0$ y $b = 0$.

Por ejemplo, la entrada podría ser

Entrada ejemplo

```

4 5
10 20
7 8
0 0

```

Salida ejemplo

```

9
30
15

```

El último par 0 0 no se procesa: solo sirve para indicar que ya no hay más casos.

```

1 int main() {
2     while (true) {
3         int a, b;
4         cin >> a >> b;
5
6         if (a == 0 && b == 0) break;
7
8         cout << a + b << "\n";
9     }
10 }

```

4.3.4. Multicaso controlado por EOF

Otro patrón clásico consiste en leer hasta encontrar el final del archivo, es decir, hasta EOF. Esto es muy común en jueces tradicionales y en problemas donde no se indica de antemano cuántos casos existen.

Por ejemplo, si la entrada es

Entrada ejemplo

```

8 2
100 50
7 13

```

Salida ejemplo

```

10
150
20

```

El programa sigue leyendo mientras todavía existan datos válidos en la entrada.

```

1 int main() {
2     int a, b;
3     while (cin >> a >> b) {
4         cout << a + b << "\n";
5     }
6 }

```

4.3.5. Multicaso con salida numerada

En este esquema se especifica el número de casos, pero además la salida debe incluir un formato como “Caso #i: respuesta”. Es importante respetar exactamente la forma pedida por el enunciado.

Por ejemplo, la entrada podría ser

Entrada ejemplo

```
3
2 3
10 1
7 8
```

Salida ejemplo

```
Caso #1: 5
Caso #2: 11
Caso #3: 15
```

Aquí no basta con imprimir el resultado: también hay que numerar cada caso según el formato exigido.

```
1 int main() {
2     int t;
3     cin >> t;
4
5     for (int caso = 1; caso <= t; caso++) {
6         int a, b;
7         cin >> a >> b;
8         cout << "Caso #" << caso << ": " << (a + b) << "\n";
9     }
10 }
```

4.3.6. Multicaso con datos en una línea

En algunos problemas, cada caso de prueba corresponde a una línea completa que contiene una cantidad arbitraria de enteros. En ese caso, además de leer, hace falta procesar la línea completa como una unidad.

Por ejemplo, si la primera línea indica cuántos casos habrá y luego cada caso es una línea con una cantidad arbitraria de enteros, la entrada podría ser

Entrada ejemplo

```
3
1 2 3
10 20
7 8 9 10
```

Salida ejemplo

```
6
30
34
```

En este patrón ya no basta con hacer `cin >> x` de forma fija, porque cada línea puede contener una cantidad distinta de números.

```
1 int main() {
2     int t;
3     cin >> t;
4     cin.ignore();
5
6     while (t--) {
7         string linea;
8         getline(cin, linea);
9
10        stringstream ss(linea);
11        int x, suma = 0;
12        while (ss >> x) {
13            suma += x;
14        }
15    }
16 }
```

```

14     }
15
16     cout << suma << "\n";
17 }
18 }

```

4.3.7. Opción de función

Finalmente, hay jueces en los que no se escribe un programa completo, sino únicamente una función ya indicada. El propio sistema se encarga de leer la entrada, construir los parámetros y llamar a esa función.

En una plataforma de este tipo, podría pensarse en un caso donde internamente el juez llama a una función con los valores 50 y 100, y espera como resultado 150. Desde la perspectiva del usuario, esa interacción muchas veces no aparece como entrada y salida estándar visibles, porque el propio sistema se encarga de esa parte.

Entrada ejemplo

50 100

(simulada por el juez)

Salida ejemplo

150

Para ilustrarlo de manera sencilla, se puede simular el comportamiento con el siguiente programa:

```

1 class Solution {
2 public:
3     int suma(int a, int b) {
4         return a + b;
5     }
6 };

```

Este formato aparece con frecuencia en plataformas orientadas a entrevistas o práctica guiada. Aunque simplifica la interacción con la entrada y la salida, no sustituye la necesidad de entender los patrones clásicos, ya que estos siguen siendo fundamentales en programación competitiva.

4.4. Una observación práctica

Aprender entrada y salida no es un tema menor ni puramente mecánico. De hecho, es una de las primeras habilidades que distinguen a quien empieza a sentirse cómodo frente a un juez en línea. Leer con precisión, identificar el patrón de los datos y respetar exactamente el formato pedido ahorra muchos errores y permite concentrarse en lo verdaderamente importante: resolver el problema.

Capítulo 5

Álgebra

El álgebra es, en esencia, el estudio de patrones numéricos, en la programación competitiva es muy común que ideas de soluciones se basen en propiedades algebraicas que veremos en esta sección.

Supongamos que queremos sumar los primeros n números naturales, claramente si N es un número pequeño como 4, rápidamente sumaríamos los números del 1 al 4 y diríamos que la respuesta es 10, sin embargo el sumar número por número no resulta práctico cuando n es un número muy grande como podría ser un millón, ¿habrá alguna forma de saber rápidamente la respuesta para cualquier n ? La respuesta es sí, llamémosle S a la suma de todos los números de 1 a n , supongamos que ponemos en una fila todos los números a sumar, es decir

$$1 + 2 + 3 + 4 + 5 + \cdots + n$$

ahora, junto a esta pongamos otra fila de los números a sumar al revés, notemos que sumarlos en un orden diferente no afecta el resultado

$$\begin{array}{ccccccc} S & = & 1 & + & 2 & + & \cdots + n \\ S & = & n & + & (n-1) & + & \cdots + 1 \end{array}$$

notemos que si sumamos cada columna tenemos que cada una suma $n+1$ y nótese que tenemos n columnas entonces si sumamos las dos listas que es $2S$ es igual que sumar las n columnas, es decir $2S = n(n+1)$ de donde tenemos que

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Esta suma es un ejemplo de **progresión aritmética**, más generalmente una progresión aritmética es una sucesión de números $\{a_1, a_2, a_3, \dots\}$ tal que cualquier término se puede obtener del anterior al sumarle una cantidad fija d . Usando la idea de arriba podemos deducir que para cualquier progresión aritmética

$$a_1 + \cdots + a_n = \frac{n(a_1 + a_n)}{2}$$

Ejercicio: Si en lugar de $1 + 2 + 3 + \cdots + n$ tuviéramos $1^2 + 2^2 + 3^2 + \cdots + n^2$ cuál sería el resultado?

Ahora supongamos que queremos saber cuánto es la siguiente suma

$$\sum_{i=0}^n r^i = 1 + r + r^2 + \cdots + r^n$$

llamemosle a la suma $S = 1 + r + r^2 + \dots + r^n$, nótese que si multiplicamos ambos lados por r tenemos que

$$rS = r + r^2 + \dots + r^{n+1}$$

ahora de ambos lados podemos restar S , es decir

$$rS - S = r + r^2 + \dots + r^n - S = r + r^2 + \dots + r^{n+1} - (1 + r + r^2 + \dots + r^n)$$

notemos que del lado derecho la mayoría de los elementos se van a cancelar, de forma que finalmente nos queda, nótese que esto tiene sentido si $r \neq 1$ de lo contrario estaríamos dividiendo entre 0.

$$S = \frac{r^{n+1} - 1}{r - 1}$$

Para $|r| < 1$ y $n \rightarrow \infty$, la serie converge:

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r}$$

Las sumas no solo sirven para obtener la respuesta del problema, también nos ayudan a evaluar complejidades, las identidades más comunes que se usan en programación competitiva son:

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2 \\ \sum_{i=0}^{n-1} r^i &= \frac{r^n - 1}{r - 1} \quad (r \neq 1) \\ \sum_{i=1}^n \frac{1}{i} &= \ln n + \gamma \quad (\text{armónica}) \end{aligned}$$

Cuadro 5.1: $\gamma \approx 0,5772$ es la constante de Euler-Mascheroni.

$\sum_{i=1}^n \lfloor n/i \rfloor \approx n \ln n$ porque $\sum_{i=1}^n n/i = n(\ln(n) + \gamma)$. Esto explica por qué iterar sobre todos los divisores de todos los números hasta n cuesta $O(n \log n)$.

Una técnica muy poderosa es cambiar el orden de una doble sumatoria. La igualdad:

$$\sum_{i=1}^n \sum_{j=i}^n f(i, j) = \sum_{j=1}^n \sum_{i=1}^j f(i, j)$$

parece trivial pero frecuentemente transforma una suma difícil en una fácil.

Ejemplo Calcular $\sum_{i=1}^n \sum_{j=i}^n \frac{1}{j}$. Cambiando el orden:

$$\sum_{j=1}^n \frac{1}{j} \sum_{i=1}^j 1 = \sum_{j=1}^n \frac{j}{j} = \sum_{j=1}^n 1 = n$$

Lo que parecía difícil se vuelve más sencillo al cambiar el orden.

5.1. Recurrencias

La sucesión $0, 1, 1, 2, 3, 5, 8, \dots$ es muy famosa y es conocida como la sucesión de fibonacci, la forma en la que se nos enseña a definirla es haciendo el primer término $F_0 = 0$, el segundo $F_1 = 1$ y desde ahí calcular los siguientes con la recurrencia

$$F_n = F_{n-1} + F_{n-2}$$

Más generalmente una recurrencia lineal se ve de la siguiente forma

$$R_x = \sum_{i=1}^n C_i R_{x_i}$$

esto significa que R_x depende de los anteriores n términos, es decir es de orden n y cada término puede tener una constante C_i multiplicandole. Por ejemplo, la recurrencia de Fibonacci es de orden 2 y la constante en cada uno de los términos es 1.

Ahora nos surge una pregunta natural, dada una recurrencia lineal de orden n , ¿cómo encontramos el término k ésimo?

Existen varias formas de solucionar este problema, y cada uno nos arroja una complejidad diferente, no es que exista una mejor o peor forma de calcular la respuesta, porque todo dependerá de qué tan grandes con k y n .

5.1.1. Solucion DP

Se le llama programación dinámica (DP por sus siglas en inglés) cuando memorizamos datos que nos ayuden a evitar hacer dobles calculos, por ejemplo en fibonacci podemos almacenar los términos que vayamos obteniendo y así cuando lleguemos al término n ya tendremos calculado $n-1$ y $n-2$ y no tendremos que volver a calcularlo. En el caso de una recurrencia lineal general la forma de implementarlo con DP es bastante simple, es como hacer fuerza bruta pero memorizando lo que ya calculamos, es decir

```
1 //necesitamos los primeros n terminos porque cada termino depende de los n anteriores
2 for(int i = n; i <= k; i++){
3     dp[i] = 0;
4     //para cada uno de los terminos anteriores j sumamos el termino multiplicado por
    su constante
5     for(int j = 1; j <= n; j++){
6         dp[i] += c[j] * dp[i-j];
7     }
8 }
```

este procedimiento tiene una complejidad $O(nk)$, es recomendable por simplicidad usarse en casos donde $nk < 1e7$ para evitar TLE.

5.1.2. Solución Exponenciación de Matrices

De la solución con programación dinámica notemos que para cada paso sólo nos importa tener la información de los últimos n términos y todos los demás, podemos olvidarlos. Si guardamos los n últimos elementos en una matriz columna, podemos aplicarle una transformación lineal para obtener los siguientes elementos, es decir si originalmente teníamos los elementos $\{R_1, R_2, \dots, R_n\}$, después

de la transformación lineal tendremos los elementos $\{R_2, R_3, \dots, R_{n+1}\}$, la transformación lineal es relativamente sencilla y consiste en multiplicar por una matriz cuadrada como sigue

$$\begin{bmatrix} C_1 & C_2 & \cdots & C_{n-1} & C_n \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix} \begin{bmatrix} R_n \\ R_{n-1} \\ R_{n-2} \\ \vdots \\ R_1 \end{bmatrix} = \begin{bmatrix} R_{n+1} \\ R_n \\ R_{n-1} \\ \vdots \\ R_2 \end{bmatrix}$$

podríamos entonces pensar que ya que cada multiplicación cambia los índices en uno, tendríamos que hacer $k - n$ multiplicaciones para obtener el término k , podríamos hacerlo, pero hacer esto nos implicaría una complejidad $O(n^3(k - n))$, en realidad podemos multiplicar la matriz cuadrada por si misma $k - n$ veces usando exponenciación binaria¹ de esta forma obtenemos el k ésimo término en tiempo $O(n^3 \log(k - n))$.

Existe otra forma de resolver de recurrencias lineales, pero necesitamos herramientas más elaboradas por lo que lo abordaremos en la sección de transformada rápida de Fourier.

5.2. Polinomios e interpolación de Lagrange

Un polinomio de grado n se representa como un vector de $n + 1$ coeficientes:

$$p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

Lo primero que nos interesara será poder evaluarlo en algún punto de forma eficiente, para esto usaremos **método de Horner**, que simplemente consiste en reescribir el polinomio como:

$$p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + x \cdot a_n) \cdots))$$

Es decir, se empezará a calcular desde el coeficiente de mayor grado y se va acumulando de adentro hacia afuera, la implementación la haremos con módulo ya que es más usada, sin embargo si no se necesita basta con quitar "mod" del código.

```
1 ll horner(vector<ll> a, ll x, ll mod) {
2     ll res = 0;
3     for (int i = a.size()-1; i >= 0; i--)
4         res = (res * x + a[i]) % mod;
5     return res;
6 }
```

Lo que queremos hacer en esta sección es dados n puntos cualesquiera, encontrar un polinomio que pasa por todos ellos, a esto se le llama interpolación de Lagrange.

Es importante mencionar que dados $n + 1$ puntos distintos $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, existe un **único** polinomio de grado a lo más n que pasa por todos ellos.

El razonamiento detrás de por qué es sencillo, supongamos que no se cumple, es decir, que hay dos polinomios diferentes p y q de grado $\leq n$ que pasan por todos los puntos dados. Entonces tendremos que $p - q$ es un polinomio de grado $\leq n$ que tiene $n + 1$ raíces (los x_i), pero sabemos que un polinomio no nulo de grado $\leq n$ tiene a lo más n raíces, de donde tenemos que tiene que ser nulo, es decir $p - q = 0$,

¹Ver sección de exponenciación binaria

por lo que $p = q$.

Esto nos dice que $n + 1$ puntos determinan completamente un polinomio de grado n . La pregunta es: ¿cómo construirlo?

La idea de Lagrange es construir el polinomio interpolante como una suma de piezas, donde cada pieza se encarga de un solo punto.

Para cada i , definimos el **polinomio base de Lagrange** $\ell_i(x)$ como el único polinomio de grado n que vale 1 en x_i y 0 en todos los demás x_j :

$$\ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Verifiquemos que cumple lo que queremos. Si evaluamos en $x = x_i$: cada factor del numerador es $x_i - x_j$ y cada factor del denominador es $x_i - x_j$, así que cada fracción vale 1 y el producto es 1. Si evaluamos en $x = x_k$ con $k \neq i$, el numerador tiene el factor $x_k - x_k = 0$, así que el producto entero es 0.

Con estas piezas, el polinomio interpolante es simplemente:

$$p(x) = \sum_{i=0}^n y_i \cdot \ell_i(x)$$

El polinomio pasa por todos los puntos pues en $x = x_i$, todos los $\ell_j(x_i) = 0$ excepto $\ell_i(x_i) = 1$, así que $p(x_i) = y_i$. Pasa exactamente por todos los puntos dados. Encontrar el polinomio nos lleva entonces tiempo $O(n^2)$.

En ocasiones podemos optimizar considerablemente este tiempo para casos en los que se nos pide calcular $f(x)$ dados $f(1), f(2), \dots, f(n)$, ya que en este caso $x_i = i$ de forma que podemos sustituir para obtener

$$p(x) = \sum_{i=0}^n f(i) \cdot \prod_{j \neq i} \frac{x - j}{i - j}$$

podríamos preguntarnos por qué esto es una optimización, nótemos que

$$\prod_{j \neq i} x - j = (x - 1)(x - 2) \cdots (x - (i - 1))(x - (i + 1)) \cdots (x - n)$$

entonces al menos para el numerador podemos precalcular los prefijos, es decir guardamos en un vector $x - 1, (x - 1)(x - 2), (x - 1)(x - 2)(x - 3), \dots, (x - 1)(x - 2) \cdots (x - n)$ y los sufijos, que es lo mismo pero del último elemento al primero, es decir $(x - n), (x - n)(x - (n - 1))$ y así hasta $(x - n)(x - (n - 1)) \cdots (x - 1)$, de forma que cuando queramos hacer la multiplicación de arriba podamos hacerlo en tiempo $O(1)$ multiplicando el prefijo $(x - 1) \cdots (x - (i - 1))$ con el prefijo $(x - n) \cdots (x - (i + 1))$.

Nótese que en el caso de no tener módulo podemos simplemente precalcular $(x - 1)(x - 2) \cdots (x - n)$ y dividir entre $(x - i)$ cuando ocupemos calcularlo para un i .

Para el denominador también podemos hacer algo similar, pues

$$\prod_{j \neq i} i - j = (i - 1)(i - 2) \cdots (i - (i - 1))(i - (i + 1)) \cdots (i - n)$$

notemos que dividiendo en dos como arriba ambas partes de la multiplicación se pueden expresar como factoriales, es decir

$$= (i - 1)!(-1)^{n-i}(n - i)!$$

para esto entonces nos interesara poder precalcular los inversos de los factoriales² y así para cada i también lo podemos calcular en $O(1)$.

Como tenemos que calcular tanto numerador como denominador para cada i , calcular $f(x)$ nos cuesta $O(n)$, lo que es considerablemente mejor que el tiempo cuadrático de arriba.

Para practicar el tema, resolvamos el siguiente problema, dados n y k tal que $0 \leq k \leq 10^6$ y $1 \leq n \leq 10^9$, calcular

$$f(n) = \sum_{i=1}^n i^k$$

Imprimir la respuesta modulo $10^9 + 7$.

Nuestro primer instinto probablemente sea calcular el resultado sumando uno a uno los términos de esta suma, sin embargo aún si pudiesemos calcular las potencias i^k de forma constante, tendríamos que sumar n términos, es decir en el peor caso 10^9 operaciones, lo que nos daría como veredicto TLE.

En realidad, podemos ver este problema desde otro enfoque, denotemos como $p_k(n) = \sum_{i=1}^n i^k$, lo primero que queremos saber es qué grado tiene el polinomio, para saber cuántos puntos necesitamos, esto lo haremos por inducción, nótese que si $k = 1$, por lo visto antes, $p_1(n) = \frac{n(n+1)}{2}$, es decir es un polinomio de grado $k + 1 = 2$, supongamos entonces que $p_{k-1}(n)$ tiene grado k , ahora consideremos $p_k(n)$ y saquemos su derivada, es decir $p'_k(n) = kp_{k-1}(n)$ de donde como la derivada baja un grado, tenemos que $p_k(n)$ tiene grado $k + 1$.

De lo anterior concluimos que necesitamos $k + 2$ puntos por los que pase el polinomio para determinarlo, estos podemos encontrarlos fácilmente pues $f(0) = 0$ y de ahí $f(x) = f(x - 1) + x^k$. Traes esto ocuparemos la interpolación vista arriba para calcular $f(n)$, de forma que la implementación nos quedaría

```
1 const ll mod = 1e9 + 7;
2 ll lagrangeInt(int n, int k) {
3     if (k == 0) return n;
4     ll res = 0;
5     if (n <= k + 1) {
6         for (int i = 1; i <= n; i++) {
7             res = (res + power(i, k)) % mod;
8         }
9         return res;
10    }
11    vector<ll> pre(k + 2), suf(k + 2), inv(k + 2);
12    inver[0] = 1;
13    pre[0] = n;
14    suf[k + 1] = n - (k + 1);
15    //precalcular prefijos
16    for (int i = 1; i <= k; i++){
17        pre[i] = pre[i - 1] * (n - i) % mod;
18    }
19    //precalcular sufijos
20    for (int i = k; i >= 1; i--){
21        suf[i] = suf[i + 1] * (n - i) % mod;
22    }
23    //precalcular inversos
24    for (int i = 1; i <= k + 1; i++){
25        inv[i] = inv[i - 1] * inv(i) % mod;
```

²Ver la sección de inversos con modulo

```

26     }
27
28     int yi = 0;
29     int num, denom;
30     for (int i = 0; i <= k + 1; i++) {
31         yi = (yi + power(i, k)) % mod;
32
33         //calcular numerador
34         if (i == 0){
35             num = suf[1];
36         }
37         else if (i == k + 1){
38             num = pre[k];
39         }
40         else{
41             num = pre[i - 1] * suf[i + 1] % mod;
42         }
43
44         //calcular denominador
45         denom = inv[i] * inv[k + 1 - i] % mod;
46
47
48         if ((i + k) % 2 == 1){
49             res += (yi * num % mod) * denom % mod;
50         }
51         else{
52             ans -= (yi * num % mod) * denom % mod;
53         }
54
55         //evitar que el resultado sea negativo
56         res = (res % mod + mod) % mod;
57     }
58     return res;
59 }
60
61 int main() {
62     int n, k; cin >> n >> k;
63     cout << lagrangeInt(n, k) << endl;
64 }

```

Ejercicio Optimizar la interpolación de lagrange como lo hicimos cuando los puntos son $1, 2, 3, \dots, n$, para cualquier progresión aritmética.

5.3. Transformada de Fourier rápida (FFT) y NTT

En la sección pasada ya trabajamos con polinomios, una duda que nos puede surgir al manipularlos es ¿cómo multiplicarlos eficientemente?, supongamos que tenemos un polinomio $A(x) = \sum_{i=0}^{n-1} a_i x^i$ y otro $B(x) = \sum_{i=0}^{n-1} b_i x^i$, el producto de ambos es $C(x) = A(x) \cdot B(x)$ y tiene coeficientes:

$$c_k = \sum_{i+j=k} a_i b_j$$

Esta operación se llama **convolución** y calcularla directamente cuesta $O(n^2)$, cuando $n \approx 10^5$, esto es 10^{10} operaciones, tiempo que nos daría TLE al intentar solucionar un problema de esta forma en la ICPC. El objetivo de *FFT* será poder hacer esta multiplicación en $O(n \log n)$.

De la sección de arriba sabemos que si tenemos un polinomio de grado $n-1$ este lo podemos determinar evaluando n puntos diferentes, basado en esto nos gustaría tener para ambos polinomios A y B sus valores en n puntos estratégicos para entonces multiplicar estos valores y obtener valores del polinomio C .

La elección de los n puntos es lo que hace eficientes el algoritmo. FFT elige las **raíces n -ésimas de la unidad** $\omega^k = e^{2\pi i k/n}$, que tienen una estructura recursiva que permite hacer la evaluación e interpolación en $O(n \log n)$ en lugar de $O(n^2)$.

Para la demostración completa del algoritmo y todos los detalles de implementación referimos al blog de Codeforces de [leanderKortmann](#), que tras leer bastantes recursos, parece ser la referencia más clara y amigable disponible para competencia.³

FFT aparece disfrazado en muchos problemas de competencia, especialmente cuando hay que contar pares con cierta suma. Reconocer que el problema se reduce a multiplicar polinomios es lo más importante en este tipo de problemas ya que usualmente basta con usar la misma plantilla de *FFT*.

FFT vs NTT: cuándo usar cada una

Hay dos variantes del algoritmo con distintos casos de uso:

	FFT	NTT
Aritmética	Números complejos (<code>long double</code>)	Números enteros mod p , p primo
Coefficientes grandes	Sí, con riesgo de error de redondeo	No, acotados por p
Respuesta exacta	Con <code>llroundl</code> si valores $\leq 10^{14}$	Siempre
Uso típico	Scores, correlaciones, valores grandes	Conteos módulo primo

Se recomienda usar *FFT* cuando los coeficientes pueden superar el módulo de *NTT* $\approx 10^9$; y *NTT* cuando la respuesta se pide módulo un primo p que se comporte bien; esto es, si $p-1$ tiene un factor de 2 suficientemente grande, garantizando raíces primitivas de la unidad módulo p para tamaños potencia de 2. El más usado en competencia es $998244353 = 119 \cdot 2^{23} + 1$, que soporta transformadas de hasta $2^{23} \approx 8 \times 10^6$ elementos.

Las implementaciones de FFT y NTT que se usan en los problemas de este capítulo se dejan aquí para referencia:

FFT

```

1
2 #include <complex>
3 using ld = long double;
4 using cd = complex<ld>;
5 const ld PI = acosl(-1.0L);
6
7 void fft(vector<cd>& a, bool inv) {
8     int n = a.size();
9     for (int i = 1, j = 0; i < n; i++) {
10         int bit = n >> 1;

```

³<https://codeforces.com/blog/entry/111371>

```

11     for (; j & bit; bit >>= 1) j ^= bit;
12     j ^= bit;
13     if (i < j) swap(a[i], a[j]);
14 }
15 for (int len = 2; len <= n; len <<= 1) {
16     ld ang = 2 * PI / len * (inv ? -1 : 1);
17     cd wlen(cosl(ang), sinl(ang));
18     for (int i = 0; i < n; i += len) {
19         cd w(1);
20         for (int j = 0; j < len / 2; j++) {
21             cd u = a[i + j];
22             cd v = a[i + j + len / 2] * w;
23             a[i + j] = u + v;
24             a[i + j + len / 2] = u - v;
25             w *= wlen;
26         }
27     }
28 }
29 if (inv)
30     for (auto& x : a) x /= n;
31 }

```

NNT

```

1  const ll MOD = 998244353;
2  const ll PRIM = 3;
3
4  void ntt(vector<ll>& a, bool inv) {
5      int n = a.size();
6      for (int i = 1, j = 0; i < n; i++) {
7          int bit = n >> 1;
8          for (; j & bit; bit >>= 1) j ^= bit;
9          j ^= bit;
10         if (i < j) swap(a[i], a[j]);
11     }
12     for (int len = 2; len <= n; len <<= 1) {
13         ll w = inv
14             ? power(PRIM, MOD - 1 - (MOD - 1) / len, MOD)
15             : power(PRIM, (MOD - 1) / len, MOD);
16         for (int i = 0; i < n; i += len) {
17             ll wn = 1;
18             for (int j = 0; j < len / 2; j++) {
19                 ll u = a[i + j];
20                 ll v = a[i + j + len / 2] * wn % MOD;
21                 a[i + j] = (u + v) % MOD;
22                 a[i + j + len / 2] = (u - v + MOD) % MOD;
23                 wn = wn * w % MOD;
24             }
25         }
26     }
27     if (inv) {
28         ll ni = power(n, MOD - 2, MOD);
29         for (auto& x : a) x = x * ni % MOD;
30     }
31 }

```

En la práctica, lo que se llama desde el problema es simplemente `multiply(a, b)`, que devuelve el vector de coeficientes del producto. Las funciones `ntt` y `fft` son internas.

```

1 vector<ll> multiply_ntt(vector<ll> a, vector<ll> b) {
2     int result_size = a.size() + b.size() - 1;

```

```

3     int n = 1;
4     while (n < result_size){
5         n <= 1;
6     }
7     a.resize(n); b.resize(n);
8     ntt(a, false); ntt(b, false);
9     for (int i = 0; i < n; i++){
10        a[i] = a[i] * b[i] % MOD;
11    }
12    ntt(a, true);
13    a.resize(result_size);
14    return a;
15 }
16 vector<ll> multiply_fft(vector<ll> a, vector<ll> b) {
17     int result_size = a.size() + b.size() - 1;
18     int n = 1;
19     while (n < result_size) n <= 1;
20     vector<cd> fa(n), fb(n);
21     for (int i = 0; i < (int)a.size(); i++){
22         fa[i] = a[i];
23     }
24     for (int i = 0; i < (int)b.size(); i++){
25         fb[i] = b[i];
26     }
27     fft(fa, false); fft(fb, false);
28     for (int i = 0; i < n; i++){
29         fa[i] *= fb[i];
30     }
31     fft(fa, true);
32     vector<ll> res(result_size);
33     for (int i = 0; i < result_size; i++){
34         res[i] = llroundl(fa[i].real());
35     }
36     return res;
37 }

```

5.4. Problemas

- Cowcompany Cowmpensation (Codeforces 995F)
- Nikita and Order Statistics (Codeforces 993E)
- Running Competition (Codeforces 1398G)
- Proyecto Hali Mary ★
- Rumbo al Mundial ★

Capítulo 6

Teoría de números

En programación competitiva, el dominar o al menos tener buenas nociones de los conceptos principales de teoría de números no es un lujo, es una necesidad, el lector se dará cuenta a lo largo de este texto y en su aventura resolviendo problemas en la ICPC que muchos de los algoritmos usan alguna noción o idea de teoría de números.

6.1. Bases

Decimos que a **divide** a b (escrito $a \mid b$) si existe un entero k tal que $b = ka$. Las propiedades que más se usan en competencia son:

- Si $a \mid b$ y $a \mid c$, entonces $a \mid (b \pm c)$.
- Si $a \mid b$, entonces $a \mid bc$ para cualquier entero c .
- Si $a \mid b$ y $b \mid c$, entonces $a \mid c$.

El **máximo común divisor** de dos números a y b , $\gcd(a, b)$ por sus siglas en inglés, es el mayor entero que divide a ambos. El **mínimo común múltiplo** $\text{lcm}(a, b)$ es el menor entero positivo divisible por ambos. Se relacionan ya que

$$\text{lcm}(a, b) = \frac{a \cdot b}{\gcd(a, b)}$$

Podría parecer que esto nos da una implementación directa de lcm , pero estamos ignorando un factor que siempre hay que tener en cuenta al hacer programación competitiva y es manejar enteros muy grandes para evitar el overflow, recordemos que un tipo de dato `int` puede almacenar 32 bits y un `long long` 64 de forma que pueden almacenar números aproximadamente hasta 10^9 y 10^{18} entonces si tuviéramos el caso donde $a = 10^{10}$ y $b = 10^9$, tendríamos que $ab = 10^{19}$ que no cabe en un `int` ni `long long`, por lo que se produciría un error llamado overflow, es por eso que el orden recomendado para realizar las operaciones es $a/\gcd(a, b) * b$, es decir, primero dividir para hacer el número menor.

El algoritmo de Euclides calcula $\gcd(a, b)$ eficientemente usando la identidad:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

La idea es que cualquier divisor común de a y b también divide a $a \bmod b = a - \lfloor a/b \rfloor \cdot b$, y viceversa. Se repite hasta que el residuo es 0, momento en que el divisor es el último valor no nulo, esto nos deja una implementación muy corta y directa:

```

1 long long gcd(long long a, long long b) {
2     return b == 0 ? a : gcd(b, a % b);
3 }
4
5 long long lcm(long long a, long long b) {
6     return a / gcd(a, b) * b;
7 }

```

La complejidad es $O(\log \min(a, b))$. En C++17 puedes usar directamente `gcd`.

6.2. Aritmética modular

Iniciemos analizando un ejemplo.

Ejemplos [3.1] Supongamos que hoy es Lunes, ¿qué día de la semana será dentro de 300 días?

Solución. Podríamos hacer una lista de 300 números después del lunes, asignando cada día a cada número y ver cuál queda en el número 300, pero esto sería muy tardado y realmente innecesario, pues si notamos que cada 7 días se repite el mismo día, podemos ver que ya que al dividir 300 entre 7 nos da 42 y nos queda de residuo 6, por lo que sabemos que el día 264, es decir 42 veces 7, será Lunes y sólo tenemos que ver los restantes 6 días, que sabiendo que 6 días después de Lunes es Domingo, tendremos nuestra respuesta.

En este ejemplo vemos como la noción de que conjuntos de números se conformen de forma cíclica nos simplifica problemas. Si en el ejemplo anterior en lugar 300 días fueran 314 días, la respuesta sería la misma, podría parecer una afirmación fuerte, pero realmente al dividir 314 entre 7 como hicimos en el ejemplo notaremos que el residuo es el mismo, es decir 6, entonces el día 308, es decir 44 veces 7, será Lunes y tenemos nuevamente que ver los siguientes 6 días, esto nos lleva a pensar que si de días de la semana estamos hablando, dos días representan el mismo día de la semana si y sólo si tienen el mismo residuo al dividirlos entre 7.

Resulta que esto mismo lo podemos hacer con cualquier número natural n , sea n un número natural. Si x y y son dos números enteros tales que tienen mismo residuo al dividirlos entre n diremos que x es congruente a y módulo n lo que podemos escribir con la siguiente notación:

$$x \equiv y \pmod{n}$$

Antes de ver ejemplos, veamos algunas propiedades de las congruencias, si $a \equiv b \pmod{n}$ y $c \equiv d \pmod{n}$ y m es un entero positivo, entonces lo siguiente se cumple:

$$a + c \equiv b + d \pmod{n}$$

$$ac \equiv bd \pmod{n}$$

$$a^m \equiv b^m \pmod{n}$$

Como podemos notar, las congruencias respetan las sumas, productos y potencias como estamos acostumbrados, y en realidad para sumas y productos ocurre que cualquier número puede sustituirse por otro al que sea congruente en el módulo de la congruencia.

Nótese que esto sirve justamente para evitar overflow, pues si estamos trabajando en algún módulo m podemos sacar la congruencia de los números en cada paso para que no se vuelvan muy grandes, en C++, dado un a , podemos calcularlo escribiendo ' $a \% m$ '. Un uso que podemos darle es realizar exponenciación rápida, usualmente cuando hacemos potencias a^b tenemos dos caminos ingenuos, uno

es usar $\text{pow}(a, b)$ esto no se recomienda por errores de precisión y tiempo, el otro es directamente multiplicar a b veces, sin embargo esto cuesta $O(b)$. Usando la identidad:

$$a^b = \begin{cases} (a^{b/2})^2 & \text{si } b \text{ es par} \\ a \cdot a^{b-1} & \text{si } b \text{ es impar} \end{cases}$$

se obtiene un algoritmo $O(\log b)$. También se puede usar esta misma identidad para hacer exponenciación rápida dado un módulo:

```
1 long long power(long long a, long long b, long long mod) {
2     long long res = 1;
3     a %= mod;
4     while (b > 0) {
5         //si b es impar se multiplica una vez extra
6         if (b % 2) res = res * a % mod;
7         a = a * a % mod;
8         b = b/2;
9     }
10    return res;
11 }
```

6.3. Números primos y criba de Eratóstenes

Un número $p > 1$ es **primo** si sus únicos divisores son 1 y p . Todo entero $n > 1$ se puede escribir de forma única como producto de primos (teorema fundamental de la aritmética).

Para verificar si n es primo basta revisar divisores hasta $\sqrt{n}$, si ninguno divide a n , entonces es primo. Esto es $O(\sqrt{n})$.

```
1 bool prime(long long n) {
2     if (n < 2) return false;
3     for (long long i = 2; i * i <= n; i++)
4         if (n % i == 0) return false;
5     return true;
6 }
```

Para encontrar todos los primos hasta n , la **criba de Eratóstenes** es el método estándar. La idea central es tener una tabla de números y tachar los múltiplos de los números que ya sabemos que son primos, comenzando siempre por el 2, es decir, se toma el 2, se declara número primo y se eliminan todos los múltiplos de ese número (como 4,6,8,...), cuando finalicemos, tomamos el siguiente en este caso 3 y repetimos el proceso hasta que el cuadrado del número actual supera el límite de la lista. Este método funciona porque si un número no ha sido tachado por ningún primo más pequeño, es imposible que sea divisible por otros, por lo que debe ser primo.

```
1 vector<bool> prime(n+1, true);
2 prime[0]=prime[1]=false;
3 for(int i = 2; i <= n; i++) {
4     //tomamos el siguiente primo
5     if(prime[i] && i*i <= n) {
6         //revisamos los multiplos
7         for(int j = i * i; j <= n; j+= i){
8             prime[j]=false;
9         }
10    }
11 }
```


La complejidad es $O(n \log \log n)$, prácticamente lineal. Funciona bien hasta $n \approx 10^7$ con memoria razonable, si se necesitan los primos hasta 10^8 o el menor factor primo de cada número (útil para factorizar rápido), existe una criba lineal $O(n)$ que calcula el menor factor primo de cada entero. Se puede consultar en cp-algorithms.com.

6.4. Teorema de Bézout

Para cualesquiera enteros a y b no ambos cero, existen enteros x e y tales que:

$$ax + by = \gcd(a, b)$$

Los coeficientes x e y se llaman **coeficientes de Bézout** y no son únicos: si (x_0, y_0) es una solución, entonces $(x_0 + k \frac{b}{\gcd}, y_0 - k \frac{a}{\gcd})$ también lo es para cualquier entero k .

Una consecuencia directa: la ecuación $ax + by = c$ tiene solución entera si y sólo si $\gcd(a, b) \mid c$.

Los coeficientes de Bézout se calculan con el **algoritmo de Euclides extendido**, el algoritmo de Euclides calcula $\gcd(a, m)$. Su versión extendida además encuentra enteros x, y tales que

$$ax + my = \gcd(a, m)$$

La idea del algoritmo extendido es seguir los mismos pasos que Euclides pero rastrear cómo se expresa cada residuo como combinación lineal de a y m . En cada paso de la división $r_{i-1} = q_i \cdot r_i + r_{i+1}$ se actualizan los coeficientes:

$$x_{i+1} = x_{i-1} - q_i \cdot x_i \quad y_{i+1} = y_{i-1} - q_i \cdot y_i$$

partiendo de $(x_0, y_0) = (1, 0)$ para a y $(x_1, y_1) = (0, 1)$ para m .

```

1 // calcula gcd(a,b) y encuentra x,y tales que ax + by = gcd(a,b)
2 long long gcdExt(long long a, long long b, long long &x, long long &y) {
3     //Caso base pues gcd(a,0)=a
4     if (b == 0) {
5         x = 1; y = 0;
6         return a;
7     }
8     long long x1, y1;
9     long long g = gcdExt(b, a % b, x1, y1);
10    x = y1;
11    y = x1 - (a / b) * y1;
12    return g;
13 }
```

La complejidad es la misma que Euclides: $O(\log \min(a, b))$.

6.5. Pequeño teorema de Fermat

Si p es primo y $\gcd(a, p) = 1$, entonces:

$$a^{p-1} \equiv 1 \pmod{p}$$

Multiplicando ambos lados por a^{-1} se obtiene $a^{p-2} \equiv a^{-1} \pmod{p}$, lo que da un método directo para calcular el inverso modular cuando el módulo es primo.

Una generalización importante es el **teorema de Euler**: si $\gcd(a, m) = 1$ entonces $a^{\phi(m)} \equiv 1 \pmod{m}$, donde $\phi(m)$ es la función de Euler (cantidad de enteros en $[1, m]$ coprimos con m). El pequeño teorema de Fermat es el caso particular $m = p$ primo, donde $\phi(p) = p - 1$.

6.6. Inverso multiplicativo modular

Aunque pueda parecer que las operaciones con congruencias son muy flexibles, hay que tener cuidado, pues aún no hemos mencionado nada sobre la división. En aritmética ordinaria, el inverso multiplicativo de un número a es $\frac{1}{a}$, es decir, el número tal que $a \cdot \frac{1}{a} = 1$. En aritmética modular necesitamos algo análogo: dado a y un módulo m , queremos encontrar un entero a^{-1} tal que

$$a \cdot a^{-1} \equiv 1 \pmod{m}$$

Esto es útil en el ICPC cada vez que necesitamos “dividir” dentro de una operación modular. Como la división directa no está definida en módulo m , la reemplazamos por multiplicar por el inverso:

$$\frac{a}{b} \pmod{m} = a \cdot b^{-1} \pmod{m}$$

Por ejemplo, al calcular combinaciones $\binom{n}{k} \pmod{p}$ necesitamos dividir factoriales, lo cual se hace multiplicando por sus inversos modulares.

Es importante mencionar que este inverso no siempre existe. Tomemos por ejemplo el caso de $m = 4$ y $a = 2$, es fácil ver que revisando todas las posibilidades de b módulo n no existe un número que cumpla lo pedido.

El inverso de a módulo m existe si y sólo si $\gcd(a, m) = 1$, es decir, a y m son coprimos. Esto se puede probar fácilmente, queremos $ax \equiv 1 \pmod{m}$, es decir, $ax - my = 1$ para algún entero y . Por el teorema de Bézout, esta ecuación tiene solución si y sólo si $\gcd(a, m) \mid 1$, lo que equivale a $\gcd(a, m) = 1$.

Ahora, también podemos diseñar un algoritmo para encontrarle en caso de que exista, arriba vimos el algoritmo de euclides extendido, si lo aplicamos a a y m , tenemos que como $\gcd(a, m) = 1$, esto nos da directamente $ax + my = 1$, es decir $ax \equiv 1 \pmod{m}$. Entonces x es el inverso de a módulo m . Entonces para implementar una función que nos de el inverso basta con llamar a `gcdExt`, es decir

```
1 long long inv(long long a, long long m) {  
2     long long x, y;  
3     gcdExt(a, m, x, y);  
4     return (x % m + m) % m;  
5 }
```

Nótese que al final en lugar de devolver x , devolvemos $(x \% m + m) \% m$, esto es porque el coeficiente x que devuelve `gcdExt` puede ser negativo. Por ejemplo, el inverso de 3 módulo 7 es 5, pero el algoritmo puede devolver $x = -2$, que es equivalente pues $-2 \equiv 5 \pmod{7}$. La expresión $(x \% m + m) \% m$ convierte cualquier x a un número congruente a x positivo en $[0, m)$.

Cuando $m = p$ es primo, el pequeño teorema de Fermat dice que $a^{-1} \equiv a^{p-2} \pmod{p}$, entonces podemos calcular el inverso con exponenciación rápida en $O(\log p)$.

Inversos para cada número módulo m

En ocasiones queremos calcular el inverso modular para cada número $1, 2, \dots, m-1$, si realizamos el algoritmo que describimos anteriormente nótese que esto tendría complejidad $O(m \log m)$, si el módulo m es primo podemos construir un algoritmo que tenga complejidad $O(m)$. Tomemos i un número en el intervalo $[1, m-1]$, entonces

$$m \bmod i = m - \left\lfloor \frac{m}{i} \right\rfloor \cdot i$$

podemos sacar módulo m de ambos lados y multiplicar por $i^{-1} \cdot (m \bmod i)^{-1}$ de donde el lado izquierdo queda $i^{-1} \cdot (m \bmod i)^{-1} \cdot (m \bmod i) = i^{-1}$ y el derecho $-\lfloor \frac{m}{i} \rfloor \cdot i \cdot i^{-1} \cdot (m \bmod i)^{-1} \bmod m = -\lfloor \frac{m}{i} \rfloor \cdot (m \bmod i)^{-1} \bmod m$ de donde obtenemos la ecuación

$$i^{-1} = -\lfloor \frac{m}{i} \rfloor \cdot (m \bmod i)^{-1} \bmod m$$

de donde la implementación es muy corta

```
1 vector<long long> inversos(int m, long long p) {
2     vector<long long> inv(m);
3     inv[1] = 1;
4     for (int i = 2; i < m; i++)
5         inv[i] = (p - (p / i) * inv[p % i] % p) % p;
6     return inv;
7 }
```

6.7. Congruencias lineales y Teorema Chino del Residuo

Congruencia lineal

Ahora, ya que sabemos calcular inversos, esto nos ayudará a solucionar congruencias lineales, es decir dados a, b, n enteros, encontrar x tal que

$$ax \equiv b \pmod{m}$$

al igual que los inversos, estas congruencias no siempre tienen solución, en particular fijemonos en el $g = \gcd(a, m)$ estas congruencias tendrán solución si y sólo si $g|b$, nótemos que además en caso de que $g = 1$ tenemos que a tiene inverso módulo m entonces podemos multiplicar por el inverso en ambos lados y así encontramos que

$$x \equiv ba^{-1} \bmod m$$

¿Qué sucede en el caso que $g|b$ pero g no es 1? Nótemos por definición $g|a$ y $g|m$ y estamos suponiendo que $g|b$ entonces existen a_g, m_g y c_g , tal que $a = ga_g$, $m = gm_g$ y $b = gb_g$ de dónde $ax - b = g(a_gx - b_g)$ de dónde m es factor de $ax - b$ si y sólo si m_g es factor de $a_gx - b_g$ de donde tenemos que $a_gx \equiv b_g \bmod m_g$, además el $\gcd(a_g, m_g) = 1$ por haber dividido entre g entonces llegamos al caso de arriba que ya vimos como resolver con inversos.

Teorema chino del residuo

Ahora veremos un resultado muy importante en teoría de números llamado **teorema chino del residuo**, una pregunta natural al trabajar con congruencias es al trabajar con sistemas de congruencias, ¿qué sucede cuando tenemos dos congruencias diferentes?, ¿cómo encontrar una x que satisfaga ambas de las ecuaciones siguientes para a_1, a_2, n_1, n_2 enteros?

$$x \equiv a_1 \bmod n_1$$

$$x \equiv a_2 \bmod n_2$$

nótese que estas congruencias implican que

$$x = a_1 + n_1b_1$$

$$x = a_2 + n_2 b_2$$

es decir que

$$a_1 + n_1 b_1 = a_2 + n_2 b_2$$

$$n_1(-b_1) + n_2 b_2 = a_1 - a_2$$

denotemos $d = \text{mcd}(n_1, n_2)$, por definición, divide el lado izquierdo, entonces si x existe, d también divide el lado derecho, es decir $a_1 - a_2$, ahora, ya sabemos que podemos encontrar x' y y' tal que $n_1 x' + n_2 y' = d$ con el algoritmo de euclides extendido, también podemos multiplicar ambos lados por $\frac{a_1 - a_2}{d}$ y tenemos

$$n_1 x' \frac{a_1 - a_2}{d} + n_2 y' \frac{a_1 - a_2}{d} = a_1 - a_2$$

de donde sabemos que $b_1 = -\frac{a_1 - a_2}{d} x'$ y $b_2 = \frac{a_1 - a_2}{d} y'$ y entonces podemos simplemente substituir b_1 para tener que

$$x = a_1 + n_1 \frac{a_2 - a_1}{d} x'$$

esto nos da una solución al sistema y en realidad, no solamente una, sean x_1 y x_2 dos soluciones diferentes, ya que $x_1 \equiv x_2 \pmod{n_1}$ y $x_1 \equiv x_2 \pmod{n_2}$, tendremos que esto equivale a que $x_1 \equiv x_2 \pmod{\text{mcm}(n_1, n_2)}$ entonces todas los enteros congruentes a x_1 módulo $\text{mcm}(n_1, n_2)$ serán soluciones, lo que nos da una infinidad de soluciones.

Aunque esta implementación se ve directa, recordemos que siempre tenemos que considerar los posibles overflow que podemos tener, para evitar esto, es recomendable hacer las operaciones sacando módulo para que sean más pequeñas las cantidades.

Ahora, este algoritmo no es exclusivo de 2 congruencias, en realidad si tuvieramos k congruencias $x \equiv a_i \pmod{n_i}$ con $i = 1, \dots, k$ podemos ir aplicar el algoritmo a la primera y segunda y obtenemos una congruencia del tipo $x \equiv s \pmod{\text{mcm}(n_1, n_2)}$ y podemos unir esta con la tercera y así sucesivamente. Este algoritmo tendrá complejidad $O(k \log(\text{mcm}(n_1, \dots, n_k)))$.

6.8. Funciones multiplicativas

Una función $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}$ es **multiplicativa** si $f(ab) = f(a)f(b)$ siempre que $\text{gcd}(a, b) = 1$. Esto permite calcularla sobre cualquier entero conociendo su valor en potencias de primos, usando la descomposición de primos del número.

Función de Euler ϕ

$\phi(n)$ cuenta los enteros en $[1, n]$ coprimos con n , recordemos que dos números son coprimos si su máximo común divisor es 1. Por ser función multiplicativa cumple la propiedad de que si a y b son coprimos entonces $\phi(ab) = \phi(a)\phi(b)$, ya que todo número n se puede ver como un producto de primos $n = p_1^{k_1} p_2^{k_2} \dots p_m^{k_m}$ podemos obtener $\phi(n)$ de cualquier número calculando ϕ para cada primo $p_i^{k_i}$ en su descomposición. Entonces ahora tenemos que preguntarnos cómo es el comportamiento de ϕ con los primos y sus potencias. Nótese que si p es un primo por definición es coprimo a todos los números en $[1, p-1]$, entonces

$$\phi(p) = p - 1$$

Ahora si en lugar de p , tenemos p^k , notemos que hay p^k/p números en $[1, p^k]$ divisibles por p , es decir no coprimos con p^k de donde los podemos restarlos y obtener que

$$\phi(p^k) = p^k - p^{k-1}$$

Una propiedad importante a recordar es la **propiedad de la suma de divisores** que nos dice que si sumamos los $\phi(d)$ para todos los d divisores de n obtenemos n

$$\sum_{d|n} \phi(d) = n$$

Para la implementación podemos hacerla de forma directa empezando n y cuando encontramos divisores quitar la cantidad de numeros que lo comparten en $[1, n]$, nótese que por ser un algoritmo de factorización, tiene complejidad $O(\sqrt{n})$.

```

1 long long phi(long long n) {
2     long long res = n;
3     for (long long p = 2; p * p <= n; p++)
4         if (n % p == 0) {
5             while (n % p == 0){
6                 n /= p;
7             }
8             res -= res / p;
9         }
10    return res;
11 }

```

Ahora, como pasó con los inversos, habrá momentos en los que queramos calcular la función ϕ para todos los números de 1 a n , si realizamos el algoritmo de arriba para cada uno tendría complejidad $O(n\sqrt{n})$, en realidad podemos mejorar mucho esta complejidad haciendo una pequeña modificación a nuestro algoritmo. En el algoritmo de arriba para cada factor primo modificamos un solo numero, en este caso, usaremos una idea similar a la de la criba, para cada primo actualizaremos todos los numeros en $[1, n]$ de los que sea factor, esto nos reduce la complejidad a $O(n\log(\log(n)))$.

```

1 vector<long long> cribaPhi(int n) {
2     vector<long long> phi(n + 1);
3     //inciamos todos los resultados
4     for(int i = 0; i <= n; i++){
5         phi[i] = i;
6     }
7     for (int i = 2; i <= n; i++)
8         //revisamos si i es primo
9         if (phi[i] == i){
10            //actualizamos todos los numeros de los que es factor i
11            for (int j = i; j <= n; j += i){
12                phi[j] -= phi[j] / i;
13            }
14        }
15    return phi;
16 }

```

En una sección previa vimos el pequeño teorema de Fermat, su generalización cuando m no es primo, se le llama **Teorema de Euler** y nos dice que si a y m son primos relativos entonces

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

Nótese que de lo anterior tenemos que si a y m son primos relativos entonces

$$a^n \equiv a^{n\phi(m)} \pmod{m}$$

que podemos simplificar a

$$a^n \equiv a^{n \pmod{\phi(m)}} \pmod{m}$$

esto nos ayuda a calcular más rápido a^n cuando n es muy grande.

Función de Möbius μ

$\mu(n)$ se define como:

$$\mu(n) = \begin{cases} 1 & \text{si } n = 1 \\ (-1)^k & \text{si } n \text{ es producto de } k \text{ primos distintos} \\ 0 & \text{si } n \text{ tiene algún factor cuadrado} \end{cases}$$

Su uso principal es la **inversión de Möbius** que nos dice que si $g(n) = \sum_{d|n} f(d)$, entonces podemos invertir las funciones de la siguiente forma

$$f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Esto permite recuperar f a partir de g y viceversa. Ahora estudiemos qué sucede con la suma de μ sobre los divisores de un número como hicimos arriba con ϕ .

$$f(n) = \sum_{d|n} \mu(d)$$

Notemos que $f(1) = 1$ y $f(p^k) = 0$ pues los divisores son potencias de p o 1, cualquier potencia p^r con $r > 1$ tendrá que $\mu(p^r) = 0$ por definición de la función, entonces sólo nos falta ver qué sucede con $f(p)$, como $\mu(p) = -1$ y $\mu(1) = 1$, $f(p^k) = 0$. De aquí que como $f(n)$ es multiplicativa se cumpla que

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & \text{si } n = 1 \\ 0 & \text{cualquier otro caso} \end{cases}$$

Podemos simplificar aún más la definición con un poco de notación, $[P]$ será 1 si P es verdadero, y 0 en cualquier otro caso, por ejemplo $[gcd(4, 8) = 4] = 1$, de forma que quedaría

$$\sum_{d|n} \mu(d) = [n = 1]$$

esto nos ayudará a simplificar la notación en los siguientes ejemplos. Ahora, podría parecer que la función de Möbius es algo sin mucha aplicación, pero es una herramienta poderosa, hagamos un ejemplo. Nuestra tarea será encontrar la cantidad de pares (x, y) con $gcd(x, y) = 1$ en el intervalo $[1, n]$.

Lo primero que debemos hacer al encontrar un enunciado de este tipo es intentar expresar lo que se nos pide, notemos que este enunciado en realidad nos pide calcular

$$f(n) = \sum_{j=1}^n \sum_{i=1}^n [gcd(i, j) = 1]$$

ingenuamente podríamos pensar en recorrer todos los pares, pero esto nos llevaría complejidad $O(n^2)$ que es una complejidad muy alta, veamos como hacerlo más eficiente, con la propiedad de arriba tenemos que $[gcd(i, j) = 1] = \sum_{d|gcd(i, j)} \mu(d)$ y además en esta última suma es la suma sobre todos los

d que dividen a $\gcd(i, j)$, que es lo mismo que hacer la suma sobre todos los d en $[1, n]$ y sólo contar el resultado cuando d divide a $\gcd(i, j)$, es decir

$$\sum_{d|\gcd(i,j)} \mu(d) = \sum_{d=1}^n [d|\gcd(i,j)] \mu(d)$$

y recordemos que d divide a $\gcd(i, j)$ si y solo si d divide a i y d divide a j , entonces tenemos

$$\sum_{d=1}^n [d|\gcd(i,j)] \mu(d) = \sum_{d=1}^n [d|i][d|j] \mu(d)$$

sustituyendo tenemos que

$$f(n) = \sum_{j=1}^n \sum_{i=1}^n \sum_{d=1}^n [d|i][d|j] \mu(d)$$

ahora simplemente podemos reordenar la forma de sumar para obtener

$$\sum_{d=1}^n \mu(d) \left(\sum_{i=1}^n [d|i] \right) \left(\sum_{j=1}^n [d|j] \right)$$

y notemos que $\sum_{i=1}^n [d|i] = \sum_{j=1}^n [d|j] = \lfloor \frac{n}{d} \rfloor$ de donde tenemos

$$f(n) = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

podemos usar una criba parecida a la que hicimos con ϕ para tener calculado μ para todos los números en $[1, n]$ y simplemente hacer un ciclo de 1 a n calculando la suma, esto nos simplifica la complejidad a $O(n)$. Vamos a implementar la criba:

```

1 vector<int> cribaMu(int n) {
2     vector<int> mu(n + 1, 1);
3     vector<bool> compuesto(n + 1, false);
4     vector<int> primos;
5     for (int i = 2; i <= n; i++) {
6         //revisar si es primo
7         if (!compuesto[i]) {
8             primos.push_back(i); mu[i] = -1;
9         }
10        for (int p : primos) {
11            if (i * p > n){
12                break;
13            }
14            compuesto[i * p] = true;
15            //revisar si tiene cuadrados
16            if (i % p == 0) {
17                mu[i * p] = 0;
18                break;
19            }
20            //mu(ip)=mu(i)mu(p)=-mu(i)
21            mu[i * p] = -mu[i];
22        }
23    }
24    return mu;
25 }
```

6.9. Logaritmo discreto y Baby-step Giant-step

El **logaritmo discreto** se refiere al problema de dados a , b y m , cómo encontrar x tal que $a^x \equiv b \pmod{m}$. Este logaritmo puede no existir, por ejemplo $2^x \equiv 3 \pmod{7}$. Nótese que por fuerza bruta podemos intentar probar $x = 0, 1, \dots, m-1$ en $O(m)$, pero esto se puede hacer más eficientemente. El algoritmo **Baby-step Giant-step (BSGS)** lo resuelve en $O(\sqrt{m})$.

Consideremos la siguiente ecuación con $\gcd(a, m) = 1$

$$a^x \equiv b \pmod{m}$$

Nótese que cualquier x en $[1, m-1]$ se puede escribir como $x = i\lceil\sqrt{m}\rceil - j$ con $1 \leq i, j \leq \lceil\sqrt{m}\rceil$, de aquí que queramos resolver

$$a^{i\lceil\sqrt{m}\rceil - j} \equiv b \pmod{m}$$

ahora, como $\gcd(a, m) = 1$ tiene inverso, entonces podemos considerar

$$a^{i\lceil\sqrt{m}\rceil} \equiv ba^j \pmod{m}$$

Este algoritmo se llama baby-step giant-step porque cuando recorremos valores de i el x da un brinco muy gigante, en cambio al variar los valores de j se da brinco muy pequeños, lo que haremos será precalcular y guardar en un mapa todos los valores de $b \cdot a^j \pmod{m}$ para $j = 0, 1, \dots, \lceil\sqrt{m}\rceil$, usaremos exponenciación binaria. Después calcularemos lo mismo para cada $i = 1, 2, \dots, \lceil\sqrt{m}\rceil$, calcular $a^{i\lceil\sqrt{m}\rceil} \pmod{m}$ y buscar si está en el mapa, es decir si hay solución.

```
1 long long bsgs(long long a, long long b, long long m) {
2     a %= m; b %= m;
3     if (b == 1 || m == 1){
4         return 0;
5     }
6
7     //calculamos solo una vez el techo de la raiz
8     long long n = ceil(sqrt((double)m));
9     unordered_map<long long, long long> baby;
10
11    //baby steps guardar b*a^j para j = 0..n
12    long long aj = b;
13    for (long long j = 0; j <= n; j++) {
14        baby[aj] = j;
15        aj = aj * a % m;
16    }
17
18    //Usamos la exponenciacion binaria que vimos antes
19    long long an = power(a, n, m);
20    long long cur = an;
21    for (long long i = 1; i <= n; i++) {
22        if (baby.count(cur)) {
23            //recuperar el exponente
24            long long x = i * n - baby[cur];
25            if (x > 0){
26                return x;
27            }
28        }
29        cur = cur * an % m;
30    }
31    return -1;
32 }
```

Este algoritmo también puede usarse para el caso general (módulo no primo o $\gcd(a, m) > 1$) existe una versión extendida que reduce el problema al caso coprimo ver caso general.

6.10. Ejercicios

[3.1] Definamos $S(x)$ como la suma de dígitos de un número x en sistema decimal. Por ejemplo $S(11) = 2$, $S(1002) = 3$, $S(5) = 5$.

Diremos que un entero x es interesante si $S(X + 1) \leq S(X)$. En cada test se te dará un número n . Tu tarea es calcular cuántos números x cumplen que $1 \leq x \leq n$ y x es interesante.

Input:

La primera línea contiene un entero t ($1 \leq t \leq 1000$) — número de casos.

Después le siguen t líneas, la i -ésima línea contiene un número entero n ($1 \leq n \leq 10^9$).

Output:

Imprima t enteros, el i -ésimo entero deberá ser la respuesta para el i -ésimo caso.

Ejemplos:

Entrada	Salida
5	0
1	1
9	1
10	3
34	88005553
880055535	

[3.2] Un número $1 \leq x \leq 10^9$ es elegido. Te darán dos enteros a y b , los dos divisores más grandes del número x y también se cumple que $1 \leq a < b < x$.

Para los números dados a, b tienes que encontrar el valor de x .

Input:

Cada test contiene varios casos. La primera línea contiene un entero t ($1 \leq t \leq 10^4$), el número de casos. Luego la descripción de los casos.

La única línea de cada caso contiene dos enteros a, b ($1 \leq a < b \leq 10^9$).

Se garantiza que a, b son los divisores más grandes para algún número $1 \leq x \leq 10^9$.

Output:

Para cada caso de prueba, imprime el número x , tal que a y b son los mayores divisores del número x . Si hay varias respuestas, imprime cualquiera de ellas.

Ejemplos:

Entrada	Salida
8	6
2 3	4
1 2	33
3 11	25
1 5	20
5 10	12
4 6	27
3 9	1000000000
2500000000 5000000000	

Capítulo 7

Combinatoria y conteo

Desde inicios de la civilización el contar ha sido de utilidad, y aunque para cosas simples como contar el cambio de una compra basta con contar moneda por moneda, cuando queremos contar cosas más grandes como el número de placas posibles en la ciudad de México o la cantidad de diferentes manos de poker que puedes tener, contar una por una no es un método eficiente ni confiable pues lo más probable es que cometamos un error, es aquí donde entra la necesidad de otras técnicas que estudiaremos a continuación.

7.1. Bases

Principio de multiplicación

Supongamos que vamos a comprar un helado en una tienda donde puedes escoger cono de sabor vainilla o chocolate, helado de fresa, mango o chicle y de toppings chispas de chocolate o cereal, ¿cuántas maneras diferentes hay de elegir un helado?

Pensemos que así como podemos elegir la combinación vainilla, fresa, chispitas podemos elegir la combinación chocolate, fresa, chispitas o vainilla, mango, cereal, hay bastantes formas de combinar los sabores y notemos que el hecho de que elijamos un sabor de cono, no limita para nada nuestra elección de helado o de topping. Es así como llegamos a la conclusión que por cada sabor de cono podemos elegir cualquiera de los 3 helados y cualquiera de los 2 toppings, y cada combinación será diferente, de donde tendremos $2 \times 3 \times 2 = 12$ formas de elegir un helado.

Esto se llama **principio de multiplicación** y nos dice que si una tarea se divide en k etapas **independientes**, es decir la elección de una etapa no limita las opciones de las demás, con $n_1, n_2, \dots, n_k$ formas de realizar cada una, el número total de formas de completar la tarea es $n_1 \cdot n_2 \cdots n_k$.

Principio de adición

Ahora supongamos que en la heladería nos dicen que además de los helados, ofrecen aguas frescas de 5 sabores, si quisiéramos solamente comprar un solo artículo de la heladería, ¿de cuántas formas podríamos hacerlo? Pues bien, ya vimos que tenemos 12 helados diferentes, a esto le sumaremos las 5 opciones de agua pues al comprar un sólo artículo tenemos que elegir una de las dos opciones y así obtenemos que hay 17 formas diferentes de realizar la tarea.

Esto se llama **principio de adición** y nos dice que si una tarea puede completarse de A o de B formas, donde A y B son mutuamente excluyentes, el total es $|A| + |B|$.

Principio del complemento

Imaginemos que queremos comprar un helado que no sea de chicle, ¿de cuántas formas podemos hacerlo?, calcularlo directamente sería ver todas las opciones que no tienen chicle, a veces esta tarea es más complicada que calcular lo opuesto, es decir, vamos a contar cuántos helados diferentes tienen chicle, notemos que si el sabor de helado está fijo, tenemos 2 opciones de cono y 2 de toppings, es decir 4 opciones. Ahora podemos simplemente restar a los 12 que teníamos, la cantidad que sí tienen chicle, dejándonos con 8. Esta idea se llama **principio del complemento** y nos conviene usarla cuando los casos no deseados tienen una estructura más simple que los deseados.

Principio del palomar

En la heladería además hay una promoción para motivar la convivencia, si llegan dos personas con el mismo cumpleaños juntas, se les regalará a ambas personas un helado, ¿cuántas personas se necesitan para asegurar que alguien tendrá un helado gratis? Podríamos pensar que se necesitan muchísimas, pero en realidad solo necesitamos 367, esto porque en el peor caso cada una de las primeras 366 personas va a cumplir un día diferente (incluyendo 29 de febrero) y cuando llegue la persona número 367 como todos los cumpleaños ya tienen una persona que cumple ese día, forzosamente compartirá cumpleaños con alguien. Esto nos dice que en una escuela de 5000 estudiantes no solamente tendrá 2 personas que cumplan el mismo día, si no que pensando en el peor caso con la misma lógica de arriba, habrá 14 personas que compartan cumpleaños con alguien más. A esta idea se le llama **principio del palomar**.

7.2. Permutaciones

Supongamos que queremos escoger el nuevo CEO y VP de una lista de 5 candidatos y queremos saber de cuántas formas diferentes se puede hacer, a esto se le llama **permutación**. El número de permutaciones de tomar r elementos tomados de un conjunto de n distintos es:

$$P(n, r) = n \cdot (n - 1) \cdots (n - r + 1) = \frac{n!}{(n - r)!}$$

Esto es así porque pensemos que si tenemos n opciones para la primera elección, una vez que lo escogimos, nos queda $n - 1$ opciones y así hasta que hayamos hecho las r elecciones. Nótese que cuando $n = r$ estamos calculando de cuántas formas se puede reordenar un conjunto completo y usando la fórmula se vuelve simplemente $n!$, esto nos dice que en el ejemplo de arriba las formas de escoger al CEO y VP son $P(5, 2) = 20$.

Permutaciones con repetición

Hay casos donde los n elementos de donde elegimos no son todos distintos, veamos por ejemplo la cantidad de formas distintas en que se puede reordenar la palabra MISSISSIPPI,

Si los n elementos no son todos distintos, por ejemplo, hay n_1 copias del elemento 1, n_2 del elemento 2, etc., con $n_1 + n_2 + \dots + n_k = n$, el número de permutaciones distintas es:

$$\frac{n!}{n_1! n_2! \dots n_k!}$$

El denominador divide las repeticiones que no generan acomodos distintos, de forma que en el ejemplo de arriba notemos hay 11 letras, 1 M, 4 I, 4 S, 2 P, de donde las maneras de reordenarle son

$$\frac{11!}{1! 4! 4! 2!} = 34\,650$$

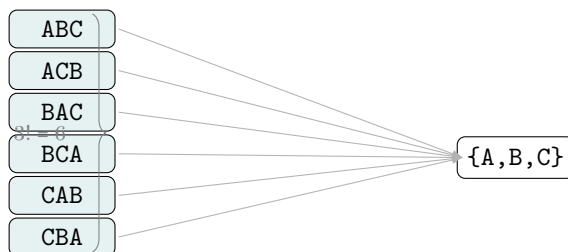
7.3. Combinaciones

Si ahora quisieramos de los 5 empleados agarrar 2 para que formen parte de una brigada no podemos aplicar directamente lo que aprendimos en permutaciones porque aquí el orden no importa, si primero elegimos al número 4 y luego al 2, es lo mismo que si elegieramos al 2 y después al 4, no hay distincion pues ambos formaran parte de la brigada. Lo que si aprendimos es que las formas de permutar un conjunto de r elementos es $r!$, entonces notemos que podemos calcular las permutaciones y luego dividir entre $r!$ que son los ordenamientos de cada selección que producen el mismo subconjunto, es decir si queremos hacer una selección de r elementos de un conjunto de n donde el orden no importa. El número de combinaciones es:

$$\binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r! (n - r)!}$$

Permutaciones

Combinaciones



A las combinaciones también se les llama coeficientes binomiales porque son los coeficientes de la expresión $(a + b)^n$ de la siguiente forma:

$$(a + b)^n = \binom{n}{0} a^n + \binom{n}{1} a^{n-1} b + \binom{n}{2} a^{n-2} b^2 + \dots + \binom{n}{n} b^n$$

Algunas propiedades útiles para recordar son (se recomienda al lector intentar probar algunas de ellas):

- Simetría: $\binom{n}{r} = \binom{n}{n-r}$
- Factorizacion: $\binom{n}{r} = \frac{n}{k} \binom{n-1}{n-1}$
- Identidad de pascal: $\binom{n-1}{r-1} + \binom{n-1}{r} = \binom{n}{r}$

- Suma sobre k: $\sum_{k=0}^n \binom{n}{k} = 2^n$
- Suma sobre m: $\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$
- Suma alternada: $\sum_{r=0}^n (-1)^r \binom{n}{r} = 0$
- Suma de cuadrados: $\binom{n}{0}^2 + \binom{n}{1}^2 + \dots + \binom{n}{n}^2 = \binom{2n}{n}$

Si solo haremos calculos rápidos sin modulo y sin numeros grandes podemos implementar directamente

```
1 ll nCr(ll n, ll k){
2     if(k>n-k)k=n-k;
3     ll comb=1;
4     for(ll i=0; i<k; i++) comb=comb*(n-i)/(i+1);
5     return comb;
6 }
```

La implementacion anterior es corta, pero es importante ser cuidadosos con la implementación pues es muy fácil obtener overflow, en general lo más usual en programación competitiva es precalcular los factoriales

```
1 long long fac[MX], invFac[MX], inv[MX];
2 void precalcComb () {
3     fac[0] = inv[0] = 1;
4     for (int i = 1; i <= MX - 1; i++) {
5         fac[i] = fac[i - 1] * i % mod;
6     }
7
8     inv[0] = inv[1] = 1;
9     for (int i = 2; i <= MX - 1; i++) {
10        inv[i] = inv[mod % i] * (mod - mod / i) % mod;
11    }
12
13    inv_fac[0] = 1;
14    for (int i = 1; i <= MX - 1; i++) {
15        invFac[i] = invFac[i - 1] * inv[i] % mod;
16    }
17 }
```

y después dependiendo de si queremos calcularlo con modulo o no utilizamos alguna de las siguientes implementaciones respectivamente

```
1 long long nCr(long long n, long long r, long long mod){
2     if (r == 0) return 1;
3     long long num = fac[n];
4     long long den = (inverseMod(fac[r],mod) * inverseMod(fac[n-r],mod)) % mod;
5     return (num * den) % mod;
6 }
7
8 long long comb (int n, int k) {
9     if (n < 0 || k < 0 || n < k) return 0;
10    return fac[n] * inv_fac[k] % mod * inv_fac[n - k] % mod;
11 }
```

7.4. Principio de Inclusión-Exclusión

En ocasiones cuando tengamos conjuntos queremos saber cuántos elementos tiene su unión, podríamos pensar que si los conjuntos son A y B basta con hacer la suma $|A| + |B|$, sin embargo, es probable que

los conjuntos no sean disjuntos, es decir, que haya elementos que pertenezcan tanto a A como B , en ese caso, les estaríamos contando doble, es por esto que en realidad $|A \cup B| = |A| + |B| - |A \cap B|$, es decir, restamos una vez los que se están contando doble.

Similarmente si tuviésemos tres conjuntos A, B, C , notemos que al sumar $|A| + |B| + |C|$ estamos contando los elementos que están en dos de los conjuntos doble como en el caso de dos conjuntos, pero no solo eso, pues los elementos que están en los tres conjuntos, están siendo contados tres veces, y al restar todas las intersecciones de dos conjuntos se están restando tres veces, por lo que habrá que sumarlos, es decir

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

usando esta misma lógica para n conjuntos nos da el llamado **principio de inclusión-exclusión**, este nos dice que si tenemos $A_1, \dots, A_n$ conjuntos entonces tenemos que

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{i=1}^n |A_i| - \sum_{i < j} |A_i \cap A_j| + \dots + (-1)^{n+1} |A_1 \cap \dots \cap A_n|$$

Esta fórmula podría parecer muy abstracta para utilizarse, por eso resolveremos un problema: Dados N, L, R queremos calcular los números x en el intervalo $[L, R]$ tal que x es coprimo con N .

Nótese que si hacemos $f(X)$ el número de x coprimos con N en el intervalo $[1, X]$, la respuesta será $f(R) - f(L - 1)$. Si bien podríamos intentar calcular la cantidad de números coprimos con N en el intervalo, es más fácil calcular lo opuesto, es decir, cuántos comparten un factor primo con N .

Con la criba podemos encontrar todos los factores primos $p_1, p_2, \dots, p_r$ de N . Ahora, sea D_i los números divisibles por p_i en el intervalo $[1, X]$, entonces $|D_i| = \lfloor \frac{X}{p_i} \rfloor$ por inclusión exclusión tenemos que:

$$|D_1 \cup D_2 \cup \dots \cup D_r| = \sum_i \lfloor \frac{X}{p_i} \rfloor - \sum_{i < j} \lfloor \frac{X}{p_i p_j} \rfloor + \sum_{i < j < k} \lfloor \frac{X}{p_i p_j p_k} \rfloor - \dots$$

y así tenemos que:

$$f(X) = X - |D_1 \cup \dots \cup D_r|$$

de forma que ya solo tenemos que implementarlo, para esto, iteraremos sobre los 2^r subconjuntos de factores primos de N . Para este tipo de problemas es importante analizar la complejidad, nótese que $2 \times 3 \times 5 \times 7 \times 11 \times 13 \times 17 \times 19 > 10^6$, de forma que si $N \leq 10^9$, este tendrá a lo más 9 factores primos, por lo que iterar sobre los $2^9 = 512$ subconjuntos sí es algo que se puede hacer rápidamente.

```

1 ll f(ll X, vector<ll>& primes) {
2     int k = primes.size();
3     ll res = 0;
4     for (int mask = 1; mask < (1 << k); mask++) {
5         ll prod = 1;
6         int bits = __builtin_popcount(mask);
7         for (int i = 0; i < k; i++)
8             if (mask >> i & 1) prod *= primes[i];
9         if (bits % 2 == 1) res += X / prod;
10        else res -= X / prod;
11    }
12    return X - res;
13 }
```

Ejercicio Completar la implementación del problema Co-prime (LCPC 2011)

7.5. Barras y estrellas

Supongamos tenemos r niños y n juguetes, ¿de cuántas formas podemos distribuir los juguetes entre los niños?, en este problema no hay restricciones, un niño puede tener 0 o todos los juguetes. Para resolver este problema usaremos "palitos" que separen los juguetes en secciones, por ejemplo, supongamos que tenemos 3 niños y 5 juguetes, entonces un acomodo donde el primer niño reciba dos juguetes, el segundo dos y tercero uno se puede representar de la siguiente forma



De la misma forma si quisiéramos r regiones, ocuparíamos $r - 1$ palitos, y notemos que podemos permutar los palitos por $(n + r - 1)!$ y después quitar las repeticiones dividiendo por $n!(r - 1)!$ así el número queda

$$\frac{(n + r - 1)!}{n!(r - 1)!}$$

esto también nos deja un resultado interesante y es que la ecuación $x_1 + x_2 + \dots + x_r = n$ donde $0 \leq x_i$ tiene una cantidad de soluciones $\frac{(n+r-1)!}{n!(r-1)!}$.

7.6. Lema de Burnside

Hay una clase de problemas de conteo donde la respuesta no es simplemente contar objetos, sino contar objetos **distintos bajo simetría**. Tomemos el ejemplo clásico: ¿cuántos collares distintos se pueden hacer con n cuentas de k colores, si dos collares son el mismo cuando uno se obtiene del otro por rotación? Contar directamente sobrecontea el mismo collar n veces (una por cada rotación). El lema de Burnside resuelve exactamente esta clase de problemas.

Para formalizar "dos objetos son el mismo si uno se obtiene del otro por alguna simetría", necesitamos el concepto de **acción de grupo**.

Tenemos dos ingredientes:

- Un conjunto X de todos los objetos que queremos contar (por ejemplo, todas las coloraciones posibles de n cuentas con k colores, hay k^n en total).
- Un conjunto G de **operaciones** (simetrías) que podemos aplicar. En el ejemplo de los collares, G son las n rotaciones posibles.

La acción de G sobre X es una función que a cada par (g, x) le asigna un nuevo objeto $g \cdot x \in X$ — el resultado de aplicar la operación g al objeto x . Para que sea una acción de grupo debe cumplir:

- La operación identidad no cambia nada: $e \cdot x = x$.
- Aplicar dos operaciones seguidas es lo mismo que aplicar su composición: $g_1 \cdot (g_2 \cdot x) = (g_1 \cdot g_2) \cdot x$.

Estas reglas básicamente dicen que la acción es compatible con la estructura del grupo y son fáciles de verificar en la práctica.

7.6.1. Órbitas y puntos fijos

La **órbita** de un objeto $x \in X$ es el conjunto de todos los objetos a los que puede llegar x al aplicarle alguna operación de G :

$$G \cdot x := \{g \cdot x \mid g \in G\}$$

Esta es la noción clave, dos objetos son *equivalentes* si están en la misma órbita. Nuestro objetivo es contar el número de órbitas $|X/G|$.

Para cada operación $g \in G$, sus **puntos fijos** son los objetos que no cambian al aplicarla:

$$X^g := \{x \in X \mid g \cdot x = x\}$$

Lema de Burnside:

El número de órbitas es igual al promedio del número de puntos fijos sobre todas las operaciones:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

La demostración completa puede consultarse en línea.¹

Usemos este lema en el caso del problema de collares de n cuentas con k colores donde dos collares son iguales si uno se obtiene del otro por rotación. En este caso, X son todas las coloraciones: $|X| = k^n$.

Nótese que si tenemos un collar con n cuentas numeradas $1, 2, \dots, n$ en círculo con k colores diferentes. La rotación por r posiciones envía la cuenta i a la posición $i + r \pmod{n}$ por lo que hay n rotaciones distintas $r = 0, 1, \dots, n - 1$ donde $r = 0$ es la identidad, es decir, no rotar. Dos formas de colorear las cuentas son las misma si alguna se obtiene de la otra bajo alguna rotación. Estas rotaciones serán precisamente nuestro grupo G que actuara sobre X .

Ahora, notemos que nos interesa saber el número de orbitas porque nos interesa la cantidad de collares diferentes y cada orbita es un collar diferente. Sabiendo esto queremos entonces aplicar el lema de burnside y para ello nos interesa saber cuántos objetos fija la rotación por r posiciones. Notemos entonces que la rotación r fija una coloración si y solo si esta es periodica con periodo $\gcd(r, n)$ ya que la cuenta i debe tener el mismo color que la cuneta $i + r$ y esto agrupa las cuentas en exactamente $\gcd(r, n)$ clases donde todas las cuentas deben tener el mismo color. Y ya que cada una puede tener k colores diferentes, tenemos que hay $k^{\gcd(r, n)}$ coloraciones fijas.

Entonces por el lema de Burnside tenemos que:

$$|X/G| = \frac{1}{n} \sum_{r=0}^{n-1} k^{\gcd(r, n)}$$

La implementación entonces queda:

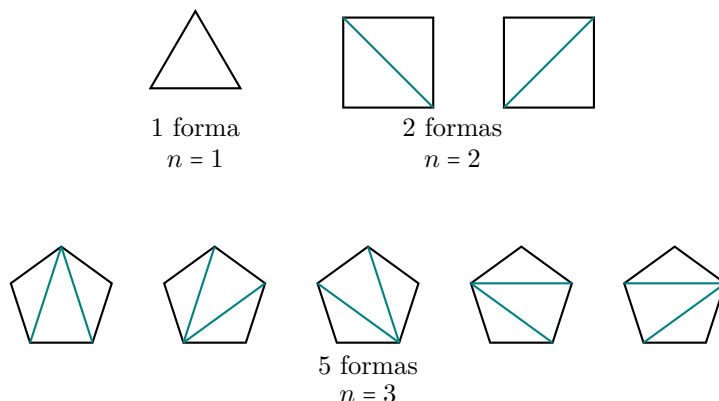
```
1 long long collares(int n, int k, long long mod) {
2     long long res = 0;
3     for (int r = 0; r < n; r++)
4         res = (res + power(k, __gcd(r, n), mod)) % mod;
5     return res % mod * power(n, mod-2, mod) % mod;
6 }
```

¹https://artofproblemsolving.com/wiki/index.php/Burnside's_Lemma

7.7. Números de Catalan

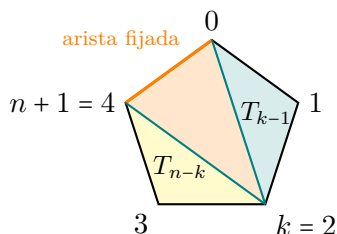
¿De cuántas formas se puede triangular un polígono convexo de $n + 2$ lados? Es decir, trazar $n - 1$ diagonales que dividan el polígono en exactamente n triángulos y que no se crucen entre sí.

Para los primeros casos podemos revisar la cantidad a mano:



Aunque pudimos hacer estos casos a mano, entre más lados tenga el polígono el número de triangulaciones se puede hacer muy grande, debido a esto es conveniente para nosotros buscar alguna otra forma de calcularlos.

Sea T_n el número de triangulaciones de un polígono de $n + 2$ lados. Fijamos la arista que va del vértice 0 al vértice $n + 1$. Esa arista pertenece a exactamente un triángulo de cualquier triangulación, llamemos k al tercer vértice de ese triángulo, con $k \in \{1, 2, \dots, n\}$. Por ejemplo para $n = 3$:



Al fijar el triángulo que contiene la arista fijada con tercer vértice $k = 2$, el polígono se divide en dos subpolígonos independientes: el subpolígono izquierdo (vértices $0, 1, \dots, k$) con T_{k-1} triangulaciones, y el derecho (vértices $k, k + 1, \dots, n + 1$) con T_{n-k} triangulaciones.

Notemos que los dos subpolígonos son independientes, es decir, la triangulación de uno no afecta al otro. Por lo tanto el número de triangulaciones con tercer vértice k es exactamente su producto $T_{k-1} \cdot T_{n-k}$. Sumando sobre todas las elecciones posibles de k , tenemos que:

$$T_n = \sum_{k=1}^n T_{k-1} \cdot T_{n-k}$$

Esta secuencia de números se llama **números de Catalan** y se denota $C_n = T_n$. Otra forma de expresar estos números es:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Nótese que estos números nos serán útiles cuando estemos resolviendo un problema en el que haya una estructura de tamaño n que se pueda dividir en dos partes independientes de tamaños $k - 1$ y $n - k$ en donde las contribuciones de cada parte se multipliquen, algunos ejemplos de problemas donde se usan son:

- Secuencias de n paréntesis balanceados.
- Árboles binarios completos con $n + 1$ hojas.
- Caminos reticulares de $(0, 0)$ a (n, n) que no cruzan la diagonal $y = x$.

7.8. Números de Stirling de segundo tipo

Los **números de Stirling de segundo tipo** $S(n, k)$ cuentan el número de formas de partir un conjunto de n elementos en exactamente k subconjuntos no vacíos.

Por ejemplo, $S(4, 2) = 7$, las 7 formas de partir $\{1, 2, 3, 4\}$ en 2 subconjuntos no vacíos son:

$$\begin{aligned} \{1\}|\{2, 3, 4\}, \quad \{2\}|\{1, 3, 4\}, \quad \{3\}|\{1, 2, 4\}, \quad \{4\}|\{1, 2, 3\}, \\ \{1, 2\}|\{3, 4\}, \quad \{1, 3\}|\{2, 4\}, \quad \{1, 4\}|\{2, 3\} \end{aligned}$$

Para calcularlos contamos con la recurrencia:

$$S(n, k) = k \cdot S(n - 1, k) + S(n - 1, k - 1)$$

con $S(0, 0) = 1$ y $S(n, 0) = S(0, k) = 0$ para $n, k > 0$.

La intuición detrás de esta recurrencia consiste en observar que el elemento n tiene 2 opciones:

- n forma un subconjunto solo, en este caso partimos $\{1, \dots, n - 1\}$ en $k - 1$ subconjuntos de $S(n - 1, k - 1)$ formas, y $\{n\}$ es el k -ésimo subconjunto.
- n comparte subconjunto con otros elementos, en este caso partimos $\{1, \dots, n - 1\}$ en k subconjuntos de $S(n - 1, k)$ formas, y agregamos n a cualquiera de los k subconjuntos existentes.

7.9. Números de Bell

El n -ésimo **número de Bell** B_n cuenta el total de particiones de un conjunto de n elementos en subconjuntos no vacíos, sin importar cuántos subconjuntos haya:

$$B_n = \sum_{k=0}^n S(n, k)$$

Es decir, B_n es la suma de números de Stirling de segundo tipo. Sin embargo, calcular todos los números de Stirling de segundo tipo para calcular los números de Bell puede ser muy costoso computacionalmente, hay una forma más eficiente de calcularlos por medio del **triángulo de Bell**, el cual es un triángulo de números que se construye así:

- La primera entrada de cada fila es igual a la última entrada de la fila anterior: $T[i][0] = T[i - 1][i - 1]$.

- El resto de la fila sigue la regla: $T[i][j] = T[i][j-1] + T[i-1][j-1]$.
- El n -ésimo número de Bell es $T[n][0]$.

1				
1	2			
2	3	5		
5	7	10	15	
15	20	27	37	52

7.10. Problemas

- Farm Game (Codeforces 1942E)
- Magic Bracelet (Spoj 2888)
- Buildings (2017-2018 ACM-ICPC GCPC)

Capítulo 8

Geometría computacional

En geometría computacional, necesitaremos una forma de representar a los objetos geométricos en código, a los puntos del plano los representaremos como un par de coordenadas de (x, y) con las operaciones usuales aplicadas coordenada a coordenada, un punto y un vector son intercambiables cuando el contexto lo permite.

```
1 typedef double db;
2 struct point {
3     db x,y;
4     point operator+(point p) {
5         return {x + p.x, y + p.y};
6     }
7     point operator-(point p) {
8         return {x - p.x, y - p.y};
9     }
10    point operator*(db a) {
11        return {x*a, y*a};
12    }
13    point operator/(db a) {
14        return {x/a, y/a};
15    }
16 }
17 };
```

Como definimos nuestra propia estructura es necesario que también definamos los comparadores para poder decidir si dos puntos son iguales o no.

```
1 bool operator==(point a, point b) {
2     return a.x==b.x && a.y==b.y;
3 }
4 bool operator!=(point a, point b) {
5     return !(a==b);
6 }
```

Nótese que el definir un comparador para decidir si un punto es "menor" a otro, no es tan fácil que hacer por lo que usualmente dependiendo el caso los ordenaremos, frecuentemente nos convendrá ordenarlos de forma horaria o antihoraria de acuerdo a los vectores que corresponden los puntos, la pregunta ahora consiste en ¿cómo hacer esto?, antes de ver una técnica para hacerlo, recordaremos algunas operaciones que podemos hacer entre puntos.

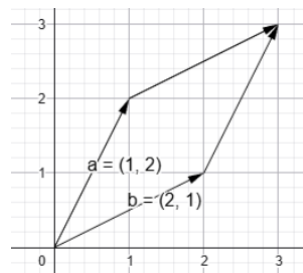
8.1. Bases

En primer lugar tenemos el producto punto, dados dos puntos este nos ayuda a medir qué tanto dos vectores apuntan a la misma dirección, si el producto punto es positivo, los vectores apuntan a direcciones similares, si es cero, los vectores son perpendiculares y si es negativo, entonces los vectores apuntan en direcciones opuestas, en programación competitiva este se calcula simplemente con las coordenadas.

```
1 double prodPunto(point a, point b){
2     return a.x*b.x + a.y*b.y;
3 }
```

Nótese que por como lo calculamos si hacemos el producto punto de un vector consigo mismo tendremos la norma de este al cuadrado.

Otro concepto que nos será relevante recordar es el producto cruz entre dos vectores (denotado $a \times b$), este nos dará el área orientada del paralelogramo que forman dos vectores, con orientada nos referimos a que si calculamos $a \times b$ será el área del paralelogramo negativa, en cambio $b \times a$ nos la dará positiva, esto nos da una noción de si un vector está a la "izquierda" o "derecha" de otro.



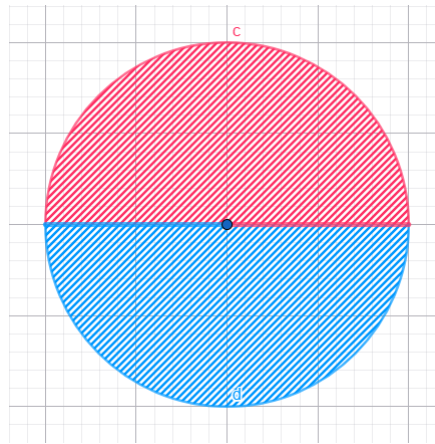
la implementación también es sencilla y nos da una forma rápida de comprobar si tres puntos son colineales.

```
1 double prodCruz(point a, point b) {
2     return a.x * b.y - a.y * b.x;
3 }
```

Con lo anterior ya tenemos las herramientas suficientes para resolver nuestro problema de ordenar los puntos del plano de forma horaria o antihoraria alrededor de un punto.

Supongamos que queremos ordenar los puntos del plano de forma antihoraria respecto al origen y con una referencia de "inicio" que tomaremos como $v = (0, 1)$.

Notemos que si tenemos un conjunto de puntos A , podemos dividir el plano en dos partes, la primera que llamaremos N que tendrá a todos los puntos que estén a la izquierda de v y todos los puntos que sean colineales apuntando a la misma dirección, la segunda se llamará S y serán los restantes, es decir, aquellos a la derecha de v y los colineales que apuntan en dirección opuesta es decir, los puntos en el área roja estarían en N y los del área azul en S .



Entonces de aquí que digamos que para un punto x en A , x está en N si y sólo si se cumple que $v \times x > 0$ o $v \cdot x > 0$ y además $v \times x = 0$, estás siendo las operaciones de producto punto y cruz que vimos arriba, con esto podemos hacer una función que nos verifique si un punto de A está o no en N fácilmente

```
1 bool N(point x){
2     point v = {1,0};
3     return prodCruz(v,x) > 0 || ( prodCruz(v,x) == 0 && prodPunto(v,x) > 0 );
4 }
```

Una vez dividido en dos partes notemos que dados x y y , podemos decir que $x < y$ si $x \in N$ y $y \in S$ o ambos están en la misma parte y además $x \times y > 0$, para esto entonces sólo nos basta definir un operador como sigue que es más fácil de manipular, nótese entonces que este ordenamiento tiene complejidad $O(n \log n)$ donde n es la cantidad de puntos en A .

```
1 bool operator <(point x, point y){
2     return (N(x) == N(y) && prodCruz(x,y) > 0) || (N(x) && !N(y));
3 }
```

8.2. Intersecciones

En muchas ocasiones nos interesará saber si dados dos segmentos AB y CD estos se intersectan, existen varias formas de revisar esto; sin embargo para evitarnos los muchos casos extremos que pueden aparecer en otros métodos, lo que haremos será pensar que en lugar de segmentos, tenemos dos líneas, la primera parametrizada como $A + t(B - A)$ y la segunda parametrizada como $(x - C) \cdot (D - C)^\perp = 0^1$, estas se van a intersectar en un solo punto siempre que no sean paralelas, lo que podemos revisar fácilmente verificando que el producto cruz sea diferente a 0, entonces, si no son paralelas, buscaremos el punto de intersección, es decir, podemos sustituir la primera recta en la segunda y obtener $(A + t(B - A) - C) \cdot (D - C)^\perp = 0$ de donde podemos despejar t

$$t = \frac{(C - A) \cdot (D - C)^\perp}{(B - A) \cdot (D - C)^\perp}$$

para sustituirlo en $A + t(B - A)$ y obtener el punto de intersección y finalmente checar si este punto está sobre cada uno de los segmentos.

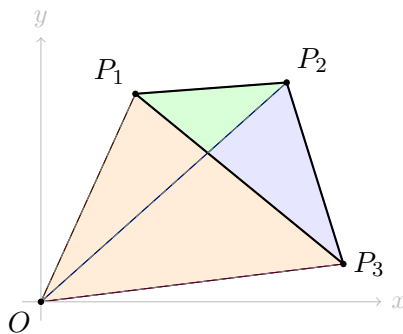
Ejercicio Implementar una función booleana que reciba cuatro puntos A, B, C y D y que nos diga si dos segmentos AB y CD se intersectan.

¹Recordemos que dado un vector (x, y) , $(x, y)^\perp = (-y, x)$, en este caso sería $(y_C - y_D, x_D - x_C)$

8.3. Polígonos

Cuando somos más pequeños, se nos enseña que el área de un triángulo se calcula como "base por altura sobre dos", imaginemos que solamente nos dan las coordenadas de los tres puntos que definen un triángulo, para usar lo que nos enseñaron tendríamos que hacer varios calculos para encontrar puntos medios y distancias, nos gustaría encontrar una forma más directa que además se pueda generalizar para polígonos en general.

Supongamos tenemos $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$, y $P_3 = (x_3, y_3)$, los tres puntos que delimitan un triángulo, notemos que si consideramos al origen $(0,0)$ entonces el area buscada equivale a sumar el área de los triángulos P_3OP_2 y P_2OP_1 y restarles el área del triángulo P_3OP_1 .



Estas áreas de los triángulos ya las sabemos calcular, pues recordemos que el producto cruz entre P_3 y P_2 nos da el área con signo del paralelogramo que forman y al dividirlo entre 2, el área del triángulo buscado es de donde deducimos que la formula del área de un triángulo es

$$A(P_1, P_2, P_3) = \frac{|P_1 \times P_2 + P_2 \times P_3 + P_3 \times P_1|}{2}$$

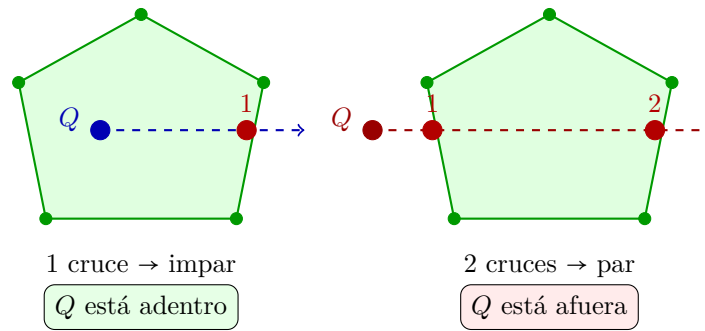
Nótese que no es necesario restar porque el producto cruz ya nos da el área con signo, entonces el signo de menos ya está incluido en la suma.

Resulta que esta misma lógica funciona para polígonos de n vértices donde se suman n triángulos haciendo que el exterior se cancele, la fórmula generalizada entonces quedaría

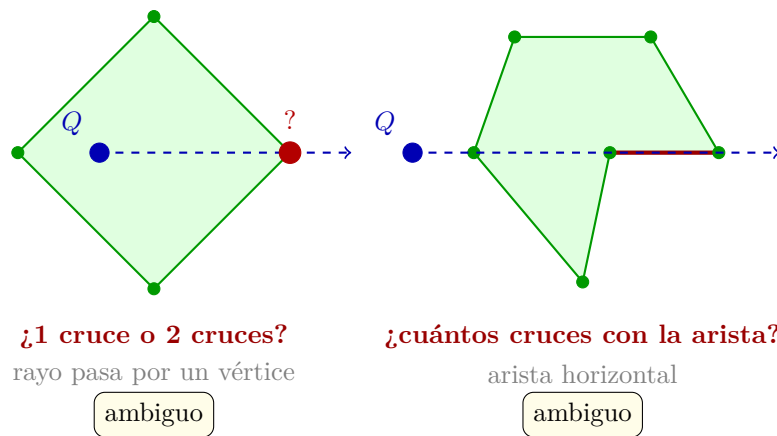
$$A(P_1, \dots, P_n) = \frac{|\sum_{i=1}^n P_i \times P_{i+1}|}{2}$$

con $P_{n+1} = P_1$. Esta fórmula funciona también para polígonos no convexos.

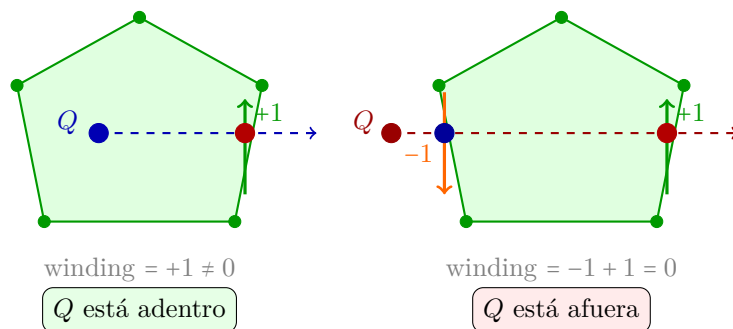
Otro problema que nos podemos encontrar al trabajar con polígonos es dado un punto Q determinar si está afuera o adentro de un polígono dado. El método más usado para resolver este problema se llama "ray casting", la idea es que se lanza un rayo horizontal desde Q hacia la derecha y contamos cuántas veces cruza el polígono, si lo cruza un número impar de veces está adentro, de lo contrario está afuera.



este es un método sencillo para problemas donde sabemos con certeza no pasarán casos extremos, ya que al lanzar el rayo podría suceder lo siguiente



Para resolver este problema, usamos un algoritmo que se llama "winding number", consiste en la misma idea de lanzar un rayo y contar intersecciones, pero en este se va a contar su orientación, nótese que en la primera la arista que intersecciona va subiendo entonces se suma 1, en el segundo caso, se resta 1 porque la arista va bajando, si el resultado es diferente de 0, el punto está adentro.



- +1 arista cruza el rayo de abajo hacia arriba
- 1 arista cruza el rayo de arriba hacia abajo

Nótese que la dirección de cada arista que cruza el rayo se determina en dos pasos: primero verificamos que los extremos de la arista estén en lados opuestos del nivel $y = y_Q$ usando las condiciones sobre las coordenadas y , y luego usamos $\text{prodCruz}(P_{i+1} - P_i, Q - P_i)$ para saber si Q queda a la izquierda o derecha de la arista, lo que determina si el cruce suma +1 o -1.

Ahora, el lector podrá preguntarse qué sucede con el caso donde el rayo intersecta exactamente en un vértice P_i . En este algoritmo las condiciones para que una arista cuente son asimétricas: para la arista $P_{i-1}P_i$ que *llega* a P_i se requiere

$$y_{P_{i-1}} \leq y_Q < y_{P_i}$$

como $y_{P_i} = y_Q$, la condición estricta $<$ no se satisface y la arista **no cuenta**. Para la arista P_iP_{i+1} que *sale* de P_i se requiere

$$y_{P_i} \leq y_Q < y_{P_{i+1}}$$

como $y_{P_i} = y_Q$, la condición $\leq$ sí se satisface, y la arista **cuenta** únicamente si $y_{P_{i+1}} > y_Q$, es decir, si la arista realmente cruza el rayo hacia arriba.

El mismo razonamiento elimina las aristas horizontales: si $y_{P_i} = y_{P_{i+1}} = y_Q$, ninguna de las dos condiciones se satisface y la arista simplemente no se cuenta, ya explicado el algoritmo, la implementación quedaría:

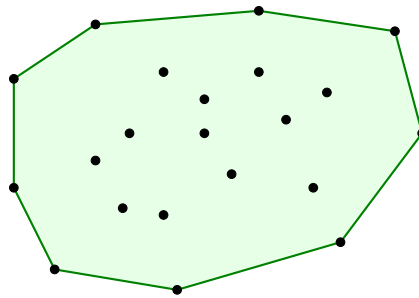
```

1 int winding_number(vector<point> poly, point q) {
2     int wn = 0;
3     int n = poly.size();
4     for (int i = 0; i < n; i++) {
5         point a = poly[i], b = poly[(i+1) % n];
6         if (a.y <= q.y) {
7             // arista cruza hacia arriba: b esta estrictamente sobre el rayo
8             if (b.y > q.y && prodCruz(b-a, q-a) > 0)
9                 wn++;
10        } else {
11            // arista cruza hacia abajo: b esta sobre o bajo el rayo
12            if (b.y <= q.y && prodCruz(b-a, q-a) < 0)
13                wn--;
14        }
15    }
16    return wn;
17 }
18
19 bool dentro(vector<point> poly, point q) {
20     return winding_number(poly, q) != 0;
21 }

```

8.4. Convex Hull

Dado un conjunto de puntos $S \subset \mathbb{R}^2$, su **cápsula convexa** (*convex hull*) intuitivamente es la figura que se formaría si colocamos una liga estirada alrededor de todos los puntos, más formalmente es el polígono convexo más pequeño que los contiene a todos. En la imagen siguiente S son todos los puntos negros y el hull sería es el polígono dibujado de verde.



Existen varios algoritmos para calcularlo. El más conocido es **Graham Scan**; sin embargo, este ordena los puntos por ángulo por lo que hay que tener cuidado con los casos delicados cuando varios puntos tienen el mismo ángulo polar. Por ello preferimos **Andrew's Monotone Chain**, ambos algoritmos tienen la misma complejidad $O(n \log n)$, misma idea para construir el convex hull, pero ordena por coordenada x en lugar de ángulo, esto lo hace más fácil de implementar sin errores en un concurso.

Nótese que para saber qué polígono es el hull, sólo tenemos que saber qué puntos forman parte de él y en qué orden. El algoritmo de Andrew construye el convex hull en dos pasadas sobre los puntos ordenados por x , primero se construye la parte inferior y luego la superior.

Para la parte superior se van a recorrer los puntos de izquierda a derecha, y tendremos un vector *hull* con la característica que los últimos tres puntos siempre van a tener producto cruz es positivo, es decir dados A, B, C tenemos que $(B - A) \times (C - A) > 0$, inicialmente se agregan los dos primeros puntos a *hull* y después cada que lleguemos a un nuevo punto P si este cumple la condición se agrega, de lo contrario, se descarta el tope de la pila y después se agrega P a *hull*. Para la parte inferior es exactamente el mismo procedimiento con la única diferencia que se recorre de derecha a izquierda. La implementación quedaría:

```

1 vector<point> convex_hull(vector<point> pts) {
2     int n = pts.size();
3     if (n < 2) return pts;
4     sort(pts.begin(), pts.end());
5     vector<point> hull;
6     /*construimos hull inferior
7     izquierda a derecha*/
8     for (int i = 0; i < n; i++) {
9         while (hull.size() >= 2 &&
10             /*producto cruz entre
11             B-A y C-A*/
12             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
13             <= 0)
14             hull.pop_back();
15         hull.push_back(pts[i]);
16     }
17     /*construimos hull superior
18     derecha a izquierda*/
19     int lower_size = hull.size() + 1;
20     for (int i = n-2; i >= 0; i--) {
21         while (hull.size() >= lower_size &&
22             /*producto cruz entre
23             B-A y C-A*/
24             prodCruz(hull.back()-hull[hull.size()-2], pts[i]-hull[hull.size()-2])
25             <= 0)
26             hull.pop_back();
27         hull.push_back(pts[i]);

```

```

27     }
28
29     //al cerrarlo este punto se repite
30     hull.pop_back();
31     return hull;
32 }

```

La desigualdad ≤ 0 descarta también los puntos colineales, de modo que el hull resultante solo contiene los vértices extremos. Si el problema requiere incluir los puntos colineales sobre las aristas del hull, podemos hacer la desigualdad estricta.

Ejercicio Dado un conjunto de n puntos, encontrar el par de puntos más lejanos (diámetro).

8.5. Algoritmos de barrido

El barrido de línea no es por sí solo un algoritmo, si no una idea general detrás de varios algoritmos. La idea es mover una línea vertical de izquierda a derecha sobre el plano y procesar los objetos geométricos conforme se van encontrando en lugar de considerar todos a la vez, a esta acción de procesamiento se le llama evento. Es decir, que en cada momento solo nos importa lo que está tocando la línea, para esto es necesario mencionar que la línea no se mueve continuamente si no que salta de evento en evento.

Estos algoritmos usualmente tienen tres partes:

- **Cola de eventos** esta es una lista ordenada por x de los eventos.
- **Estructura de incidencia** es una estructura que nos dirá qué objetos intersectan la línea en este momento, la elección de la estructura a usar depende el problema, pueden usarse sets, multisets, segment trees, colas, entre otras.
- **Procesamiento de eventos** es la lógica con la que procesaremos cada que ocurra un evento, esta depende mucho de qué problema particular querramos resolver.

Propongamos el siguiente problema: Dados n segmentos en el plano, decir si al menos dos de ellos se intersectan.

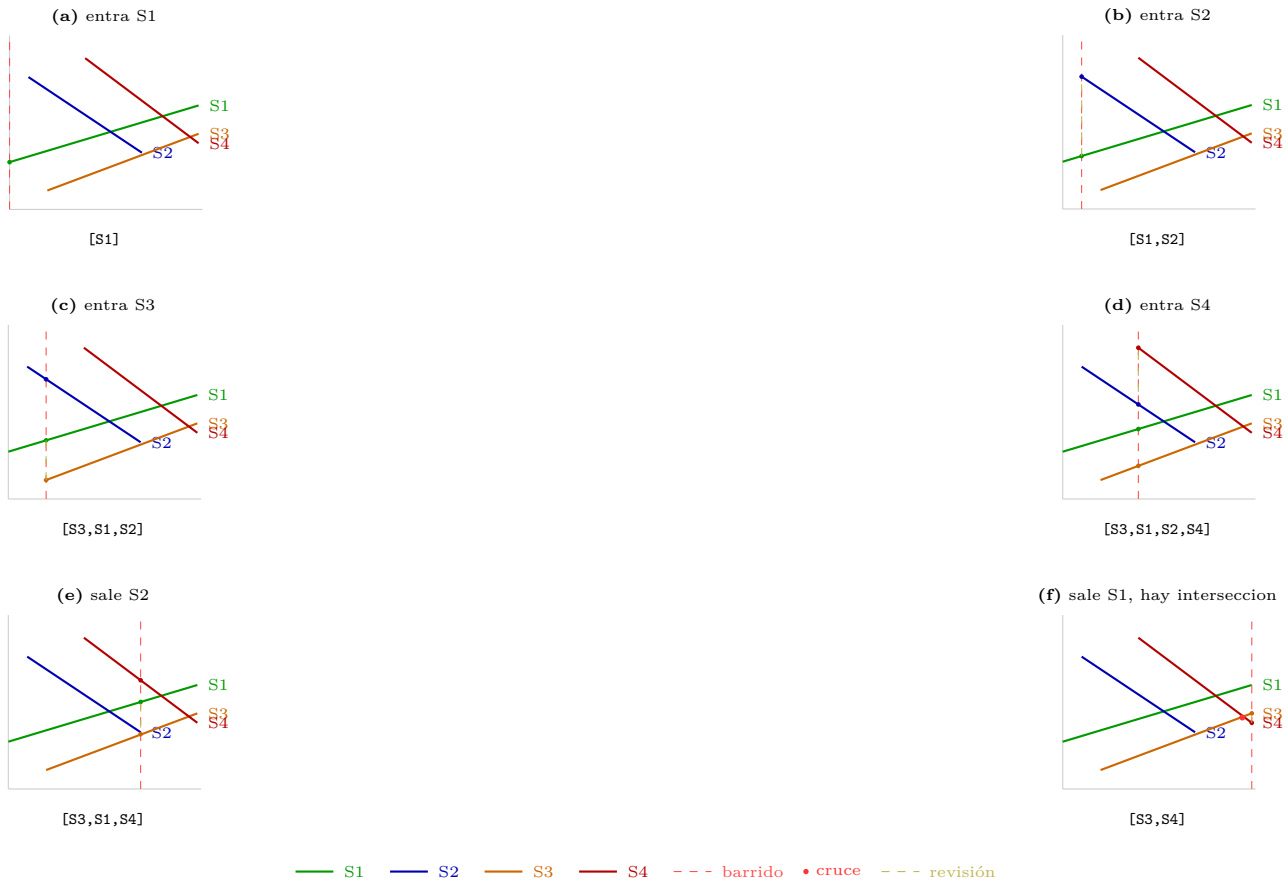
Podríamos pensar ingenuamente que la única forma de resolver este problema es revisar todos los pares de segmentos y revisar intersección entre ellos es decir una complejidad de $O(n^2)$, sin embargo, usando barrido de línea podemos resolver este problema más rápido.

El algoritmo que veremos para resolver este problema se llama *Shamos – Hoey* y se basa en que si dos segmentos se llegan a intersectar existe un momento durante el barrido en el que son vecinos en la estructura ordenada por y . Esto hace que el problema se reduzca a que en lugar de revisar todos los pares, revisar los vecinos en la estructura cada que suceda un evento.

Los eventos que podemos tener en este problema son puntos que indican el inicio o final de un segmento, entonces tendremos dos tipos de eventos:

- **Inicio** Insertaremos el punto a la estructura que ya dijimos está ordenada por y y verificaremos si se intersecta con el vecino superior o inferior.

- **Final** Este evento indica el fin de un segmento, entonces lo quitaremos de la estructura, pues ya no nos interesa. Sin embargo, hay que tener cuidado es necesario revisar si los dos vecinos (que al quitar el punto se vuelven adyacentes) se intersecan.



Si en cualquier punto encontramos una intersección, entonces terminamos el algoritmo pues ya resolvimos el problema. Es importante mencionar, que si hay varios eventos en un mismo x , primero procesaremos los inicios y después los finales, así lograremos también comprobar intersecciones si los segmentos comparten vértices.

La estructura que nos convendrá para este problema es un `set<pair<point,point>>` de C++ ordenado por la coordenada y de cada segmento en la posición actual x de la línea. El comparador para ordenar evalúa para cada segmento

$$y(x) = y_1 + \frac{(y_2 - y_1)(x - x_1)}{x_2 - x_1}$$

al implementarlo hay que ser cuidadosos pues como estaremos trabajando con decimales al hacer comparaciones debemos agregar una tolerancia ϵ .

```

1 //representamos segmentos como pares de puntos
2 using seg = pair<point, point>;
3 //funcion que nos devuelve dado un segmento y un x, la y del segmento.
4 db getY(const seg& s, db x) {
5     if (abs(s.first.x - s.second.x) < 1e-9){
6         return s.first.y;
    }

```

```

7     }
8     return s.first.y +
9         (s.second.y - s.first.y)
10        * (x - s.first.x)
11        / (s.second.x - s.first.x);
12 }
13
14 db sweepX;
15 //Comparador para ordenar los segmentos por y
16 struct cmp {
17     bool operator()(const seg& a, const seg& b) const {
18         db ya = getY(a, sweepX);
19         db yb = getY(b, sweepX);
20         if (abs(ya - yb) > 1e-9){
21             return ya < yb;
22         }
23         return a < b;
24     }
25 };
26
27 bool shamos_hoey(vector<seg> segs) {
28     int n = segs.size();
29     /*asegurarse que los segmentos
30     vayan de izquierda a derecha*/
31     for (auto& s : segs){
32         if (s.first.x > s.second.x){
33             swap(s.first, s.second);
34         }
35     }
36     /*
37     eventos guardara la x del punto,
38     1 o -1 que nos indicara si es de inicio
39     o fin respectivamente,
40     el indice en el vector original
41     */
42     vector<tuple<db,int,int>> eventos;
43     for (int i = 0; i < n; i++) {
44         eventos.push_back(
45             {segs[i].first.x, 1, i}
46         );
47         eventos.push_back(
48             {segs[i].second.x, -1, i}
49         );
50     }
51     sort(eventos.begin(), eventos.end(),
52         [](auto& a, auto& b){
53             if (abs(get<0>(a)
54                 - get<0>(b)) > 1e-9){
55                 return get<0>(a) < get<0>(b);
56             }
57             return get<1>(a) > get<1>(b);
58         });
59
60     set<seg,cmp> activos;
61     /*estructura que nos ayude
62     a borrar sin depender del
63     comparador que definimos*/
64     vector<set<seg,cmp>::iterator> pos(n, activos.end());
65     for (auto& [x, tipo, id] : eventos) {
66         sweepX = x;
67         //procesamos nodo de inicio

```

```

68     if (tipo == 1) {
69         auto it =
70             activos.insert(segs[id]).first;
71         pos[id] = it;
72         auto sig = next(it);
73         auto ant = (it != activos.begin())
74             ? prev(it)
75             : activos.end();
76         /*buscar interseccion con
77         vecinos, se dejo como ejercicio
78         en intersecciones*/
79         if (sig != activos.end()
80             && intersectan(it, sig)){
81             return true;
82         }
83         if (ant != activos.end()
84             && intersectan(it, ant)){
85             return true;
86         }
87         //procesamos nodo de final
88     } else {
89         auto it = pos[id];
90         auto sig = next(it);
91         auto ant = (it != activos.begin())
92             ? prev(it)
93             : activos.end();
94         if (sig != activos.end()
95             && ant != activos.end()
96             && intersectan(*sig, *ant)){
97             return true;
98         }
99         activos.erase(it);
100     }
101 }
102 return false;
103 }

```

Nótese que hay $2n$ eventos y cada evento realiza a lo más dos chequeos de intersección que cada uno tiene complejidad de $O(1)$ y una adición o eliminación con complejidad $O(\log n)$. Además ordenar inicialmente los segmentos tiene complejidad $O(n \log n)$ entonces la complejidad del algoritmo es $O(n \log n)$ que en efecto es más rápido que $O(n^2)$.

En el problema que resolvimos bastaba con encontrar una intersección, podríamos preguntarnos qué sucede si queremos encontrar todas las intersecciones. El algoritmo que se usa para este caso se llama **Bentley-Ottmann** que encuentra todas las intersecciones en $O((n + k) \log n)$ donde k es el número de intersecciones. Podría parecer que la diferencia es sutil y sólo basta con seguir el algoritmo que estabamos usando antes sin detenerlo, pero recordemos que nuestro algoritmo dependía fuertemente en el orden de los segmentos para encontrar intersecciones, cuando sucede una intersección, el orden de dos segmentos se cambia esto provoca que cada que encontremos un cruce tendremos que cambiar el orden de la estructura y además revisar si los nuevos vecinos tienen cruces. Y no solo eso, si no que cada que agreguemos un segmento si encontramos una intersección tendremos que calcularla y agregarla a la cola de eventos.

Mayores complicaciones de Bentley-Ottmann

- Cuando dos segmentos cambian de orden, nuestro `set` ordenado por y ya no es correcto, pues el comparador de este cambia con el tiempo dependiendo de x , tendríamos que manualmente

extraer e insertar los elementos necesarios.

- Si hay casos degenerados con tres segmentos o más que se intersectan en el mismo punto, hay que ser muy cuidadosos en que orden se procesan y cómo actualizamos la estructura usada.
- Si no sabemos con certeza una cota superior para la cantidad de intersecciones y se dé el peor caso (n^2), puede suceder que el algoritmo con complejidad $O(n^2 \log n)$ sea más lento que la fuerza bruta que era $O(n^2)$.

Usualmente no se necesita implementar Bentley Oattmann en concursos, ya que su implementación son alrededor de 200 líneas con casos delicados de manejar, sería complicado encontrar bugs bajo presión de tiempo, además que los problemas donde se requieren todas las intersecciones usualmente permiten fuerza bruta. Lo más usado es la idea en sí misma aplicado a problemas más sencillos como la unión de área de rectángulos como veremos en los ejemplos a continuación.

8.6. Ejercicios

8.7. Problemas

Bibliografía

- [1] Art of Problem Solving, *Lema de Burnside*, https://artofproblemsolving.com/wiki/index.php/Burnside's_Lemma